

Estruturas de Dados

Estruturas multidimensionais e Espaciais



2021/2022

Informação Multidimensional

- Uma questão relevante é que frequentemente a informação é de cariz multidimensional.
 - Um **Ponto** tem coordenadas x , y e z
 - Uma **Pessoa** tem um *nome*, *género*, *nº de CC*, *data de nascimento* e *morada*.
- Informação multidimensional pode ser definida como sendo um conjunto de pontos num espaço de dimensão elevada, com uma dimensão por cada atributo.
 - Cada **Ponto** existe num espaço *tridimensional*.
 - Cada **Pessoa** existe num espaço *pentadimensional* (de dimensão 5).



Informação Espacial

- Caso a informação corresponda a informação ***posicional*** contínua (*por exemplo, coordenadas de um ponto*), então:
 - Utilizam-se normalmente as mesmas unidades para cada coordenada.
 - Podem ser realizadas pesquisas relativas à *distâncias* entre objetos.
 - É frequentemente de interesse considerar também o espaço definido pela informação, e não apenas os pontos individuais:
 - Linhas (p.ex. estradas, ruas)
 - Regiões (p.ex: Lagos, cidades, países, poliedros)



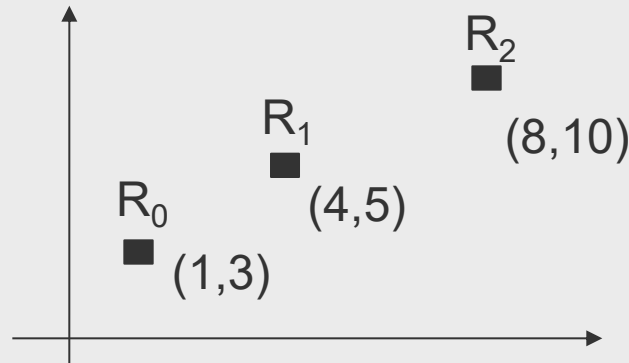
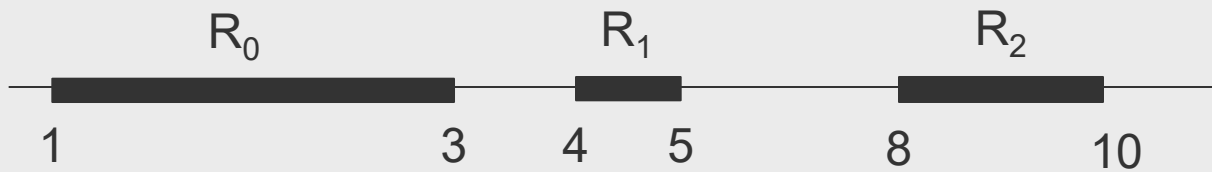
Informação Espacial

- Uma forma simples de lidar com o espaço é parametrizar e agregar a informação e *mapeá-la* (ou *transformá-la*) num objeto de dimensão mais elevada.
 - *Por exemplo, um segmento de reta num plano bidimensional definido por dois pontos pode ser representado como um único ponto num espaço de 4 dimensões.*



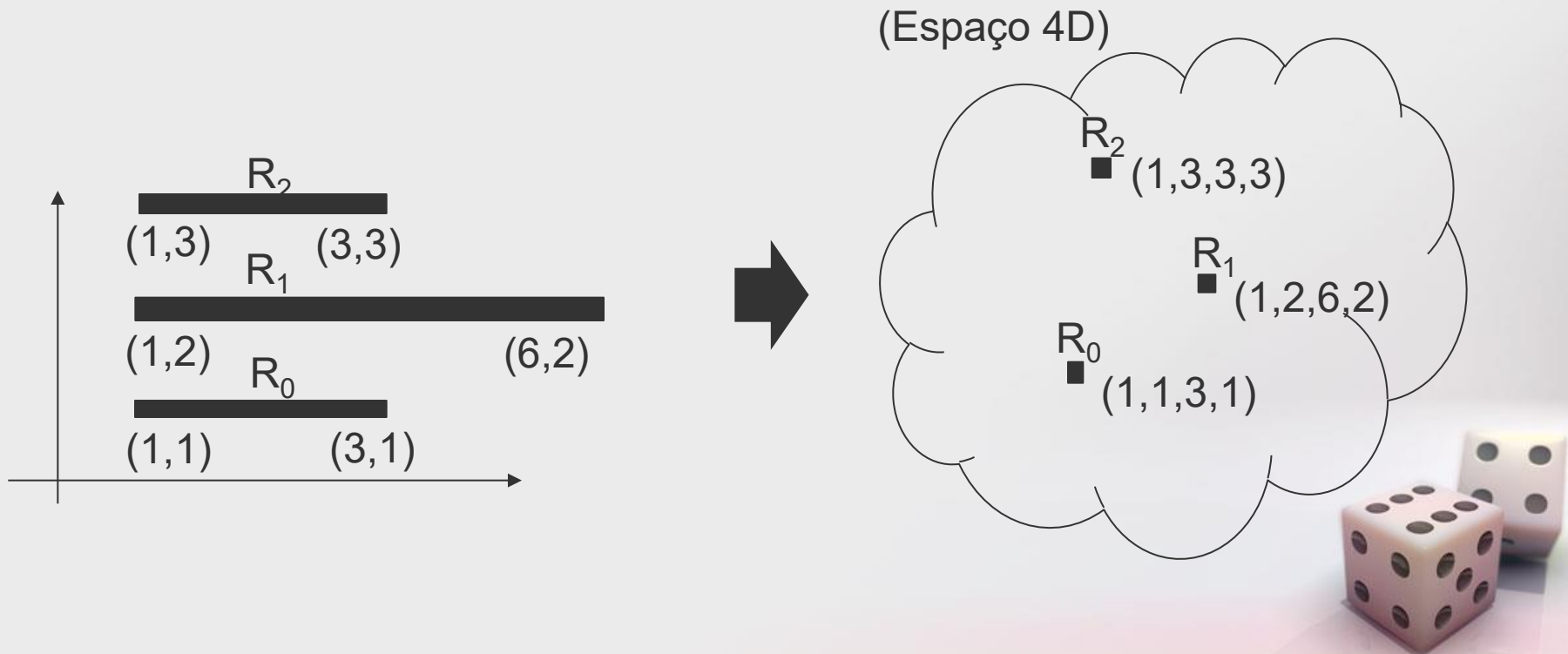
Exemplo de Transformação

- Transformação de segmentos de retas num espaço unidimensional para pontos de um espaço bidimensional.



Exemplo de Transformação

- Transformação de segmentos de reta num espaço bidimensional para pontos de um espaço tetradimensional.



Estruturas Multidimensionais

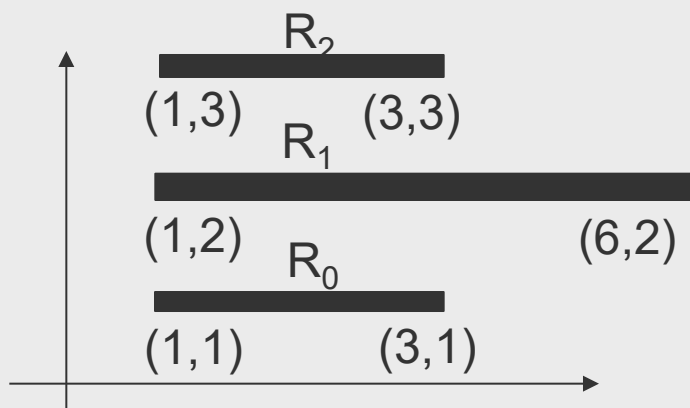
- Se a informação não possui características de localidade espacial, pode normalmente ser representada “confortavelmente” por um único ponto:
 - *P.ex. pessoa.*
- Por outro lado, as técnicas de mapeamento permitem converter informação espacial em pontos multidimensionais.

Dessa forma, se tivermos estruturas que nos permitam **armazenar pontos multidimensionais**, conseguimos representar tanto informação de natureza discreta como informação espacial (contínua). *No entanto, há um problema....*



Transformação de Informação Espacial

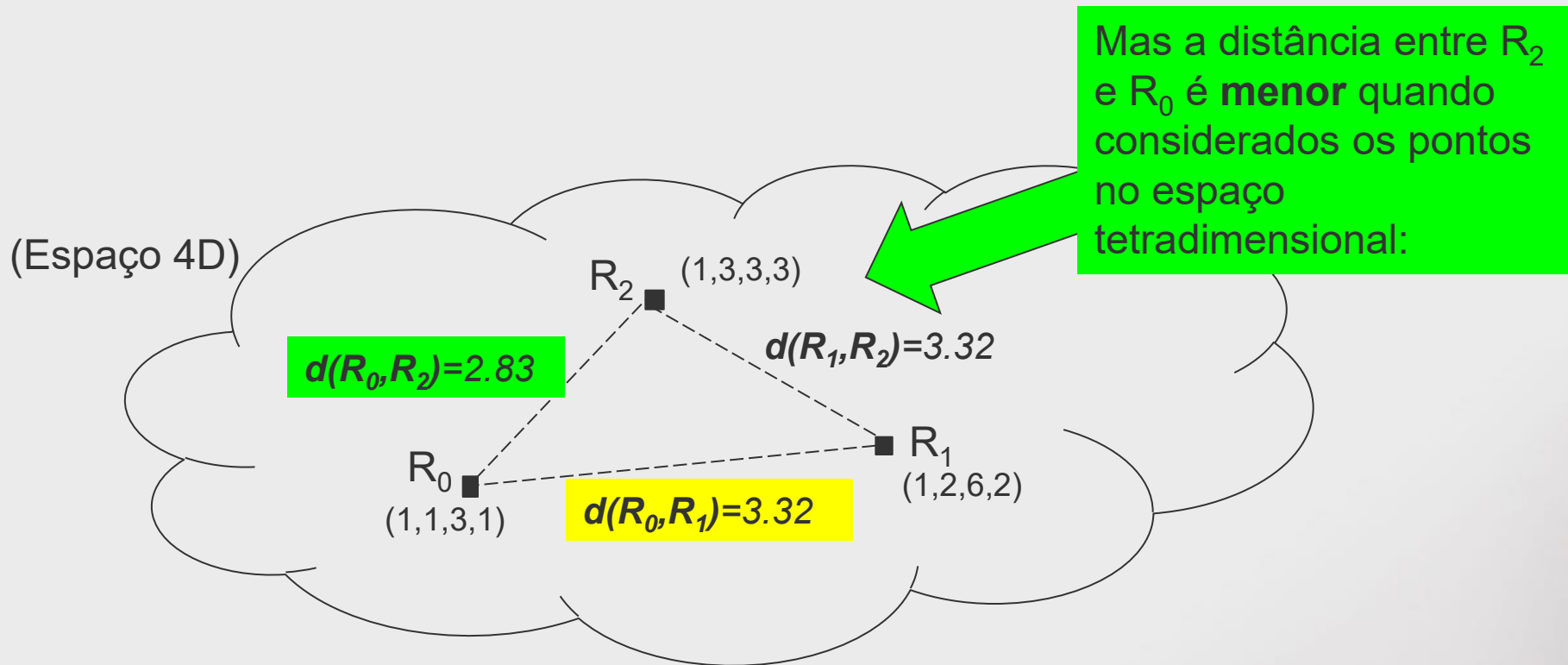
- A transformação descrita funciona bem se o objetivo é apenas consultar a informação:
 - *Quais são os segmentos de reta, que pontos definem cada segmento?*
- Mas não facilita a realização de pesquisas sobre esses objetos espaciais:
 - *Qual a reta mais próxima de outra?*



A reta mais próxima de R_0 é R_1 ...



Transformação de Informação Espacial



a distância Euclidiana entre $\mathbf{p}$ e $\mathbf{q}$ num espaço de n dimensões é dada por:

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \cdots + (p_i - q_i)^2 + \cdots + (p_n - q_n)^2} = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}.$$



Transformação de Informação Espacial

- É possível reverter a transformação e trabalhar no espaço original para obter a resposta:
 - Mas então, se é necessário reverter a transformação, para que é que ela foi feita inicialmente?...
- Tendo em conta estas limitações, vamos procurar encontrar ***soluções específicas*** para ***informação espacial***.
 - Estruturas de dados que armazenam a **ocupação de espaço** e não pontos de dados discretos multidimensionais.



Informação Multidimensional vs. Informação Espacial

- Em síntese:
 - Vamos analisar estruturas que permitem **armazenar pontos multidimensionais** e que são por isso adequadas para informação de natureza discreta (sem localidade espacial).
 - Vamos também analisar estruturas que permitem **armazenar informação espacial de natureza contínua**, que permitem não só aceder à informação mas também realizar pesquisas complexas de natureza espacial (ver se um objecto intersecta ou contém outro objeto, etc) de forma eficiente e eficaz.



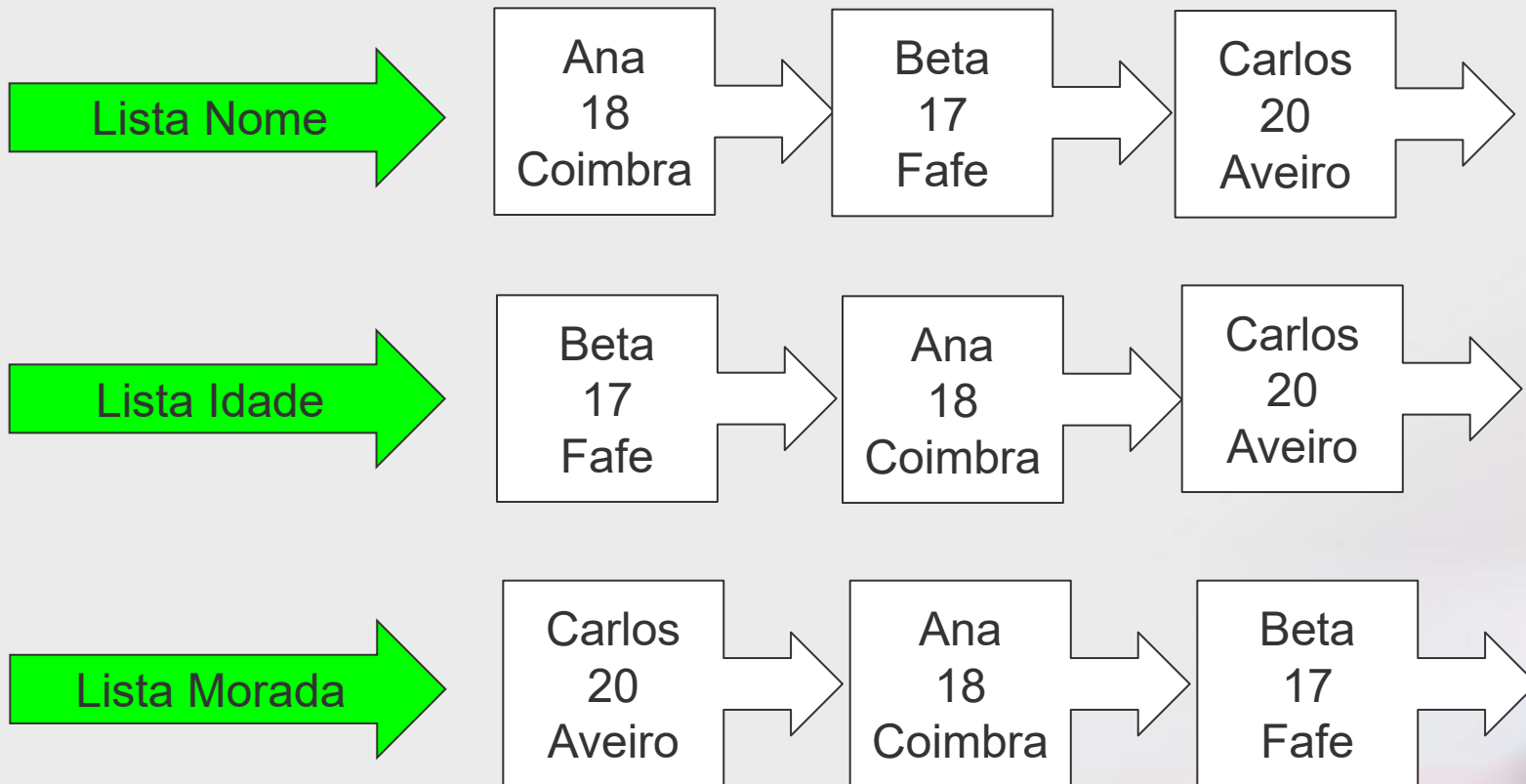
Estruturas para Pontos Multidimensionais

- A solução mais simples para armazenar pontos multidimensionais consiste na utilização de várias listas ordenadas de acordo com cada uma das dimensões.
- Esta técnica é conhecida como *listas invertidas*



Listas Invertidas

- Exemplo:** *Pretende-se armazenar informação sobre pessoas definidas por (nome, idade, morada). Neste caso seriam usadas 3 listas (uma por atributo)*



Listas Invertidas

- Só permitem acelerar a pesquisa em uma das dimensões/atributos... ☹️
- *Não é por isso particularmente útil para pesquisas mais complexas que envolvam vários atributos.*
- O método de **grelha fixa** procura ultrapassar isto...



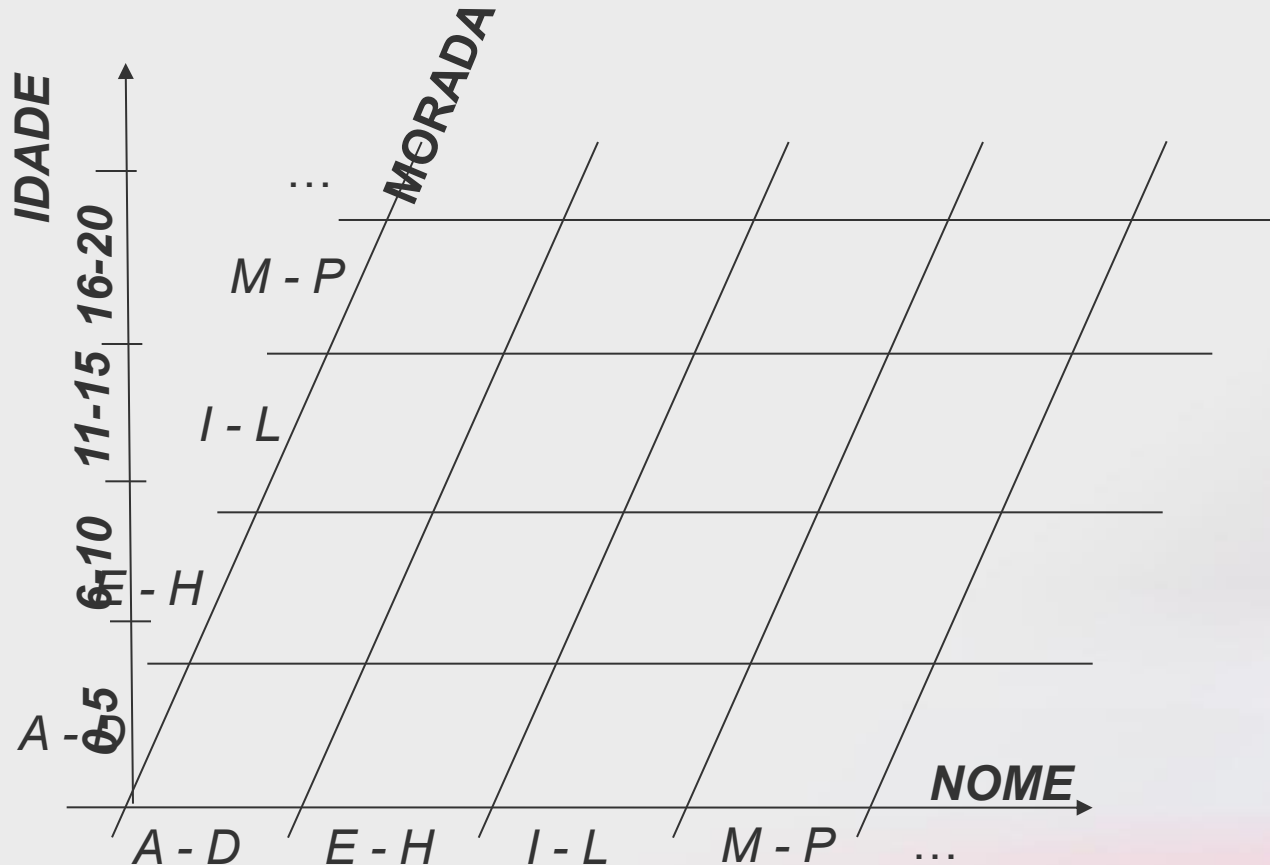
Grelha Fixa

- O princípio desta abordagem passa por:
 - Subdividir o espaço dos pontos por uma **grelha**, sendo usada uma **estrutura secundária** (*p.ex. uma lista*) por cada espaço da grelha, para armazenar os pontos nele contidos.
 - Usar um **diretório** que identifica qual o espaço ao qual pertence um ponto e que redireciona para a estrutura secundária correspondente.



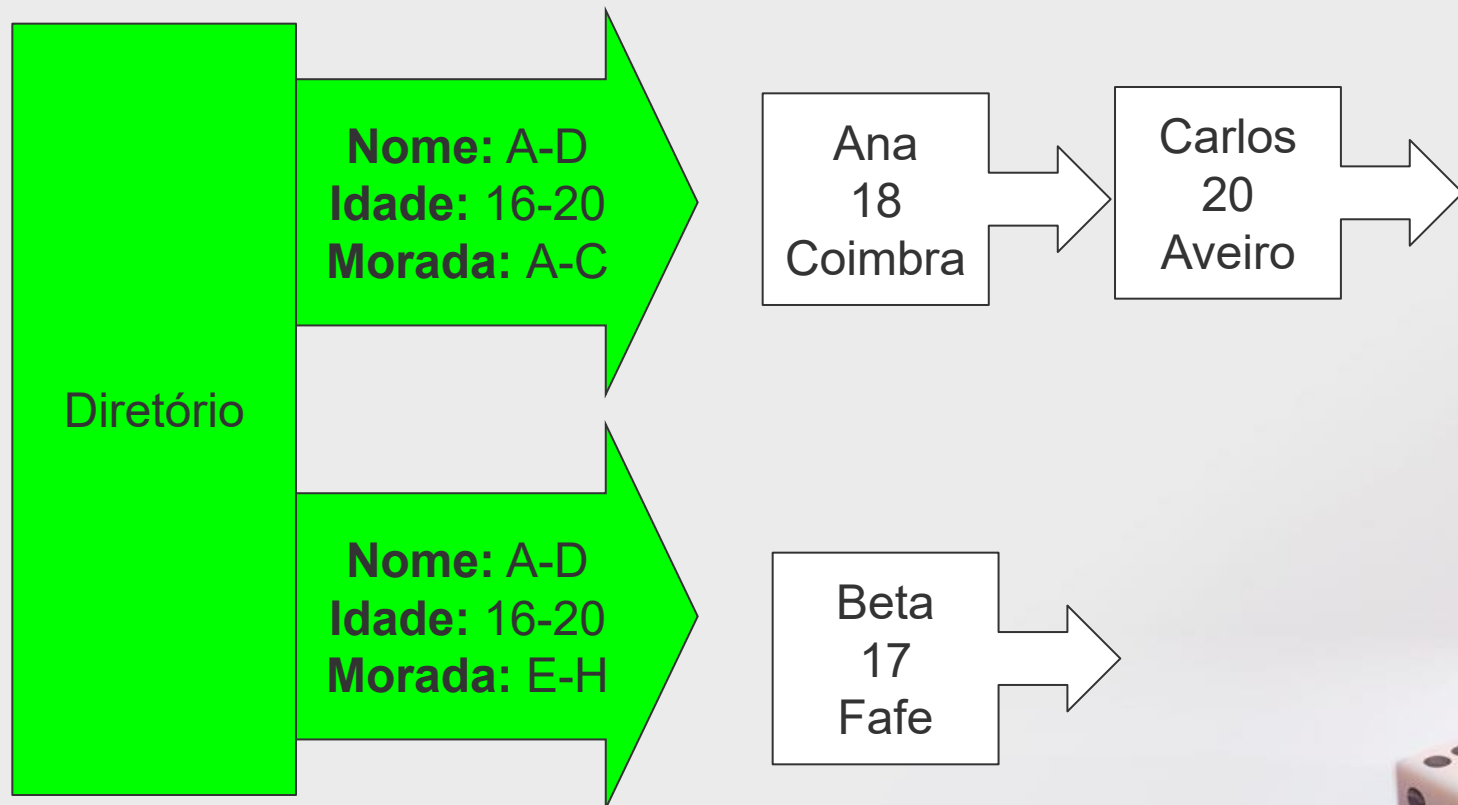
Grelha Fixa

- Exemplo:** Pretende-se armazenar informação sobre pessoas definidas por (nome, idade, morada). Um exemplo possível de grelha seria:



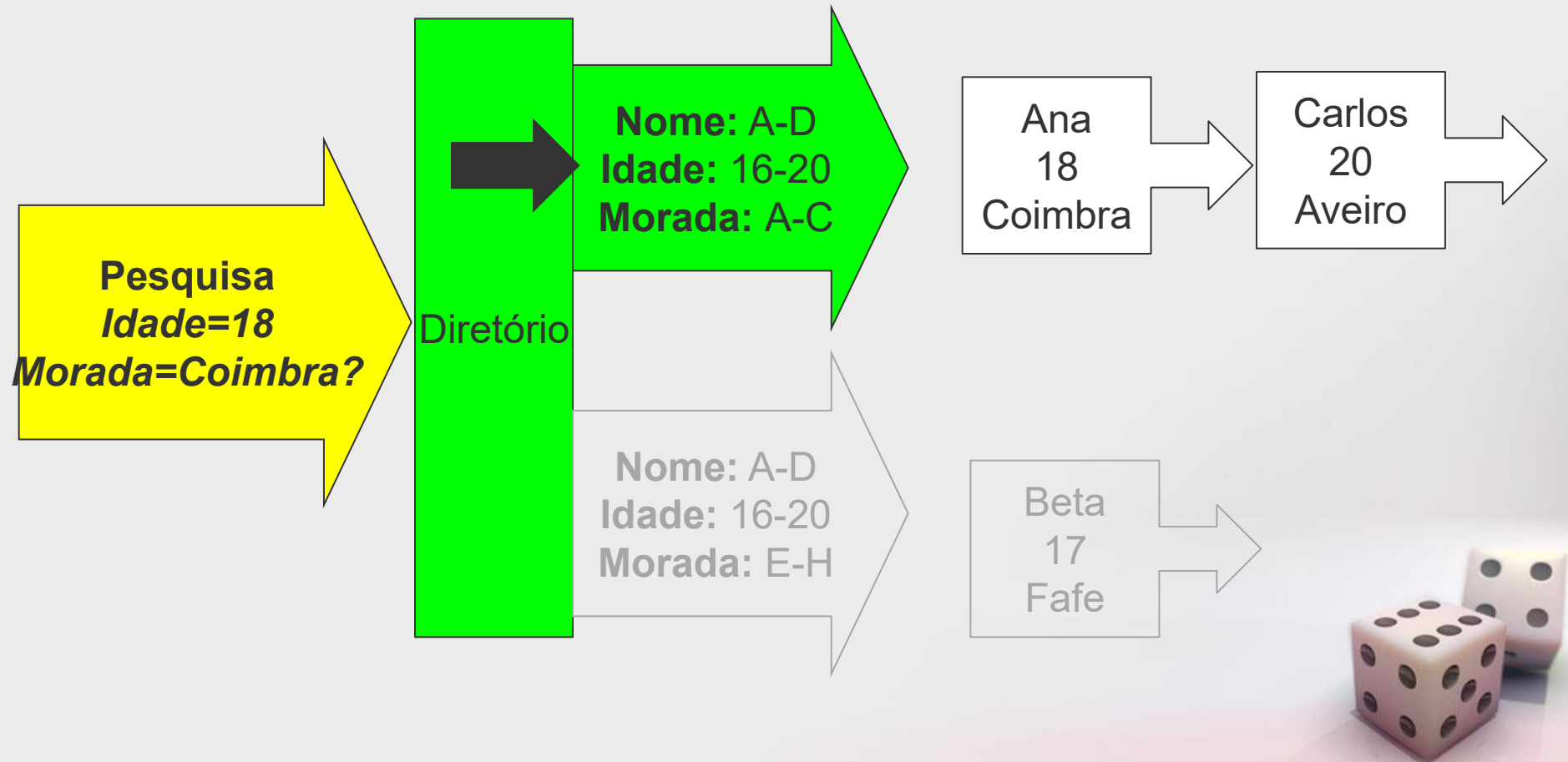
Grelha Fixa

- Exemplo (continuado):** *O diretório associa a cada espaço da grelha uma estrutura auxiliar*



Grelha Fixa

- Exemplo (continuado):** *Conforme a pesquisa desejada, o diretório identifica a lista ou listas que podem conter a informação pretendida.*



Grelha Uniforme vs. Grelha não Uniforme

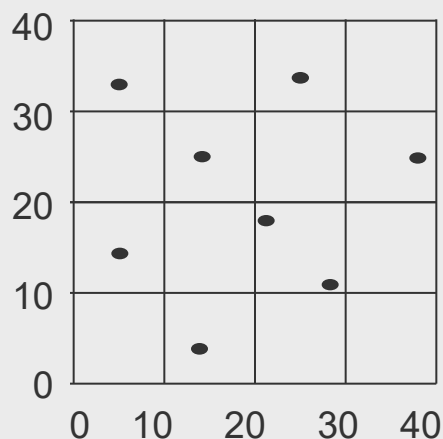
- A grelha pode ser definida de acordo com duas abordagens diferentes:
 - Dividir o **espaço** equitativamente em cada uma das dimensões. Resulta numa **grelha uniforme**
 - Pretende organizar o espaço.
 - O diretório é facilmente implementado com um *array* multidimensional.
 - Tentar dividir os **dados** equitativamente, colocando as divisões em posições arbitrárias e não regulares.
 - Pretende organizar os dados.



Grelha Uniforme vs. Grelha não Uniforme

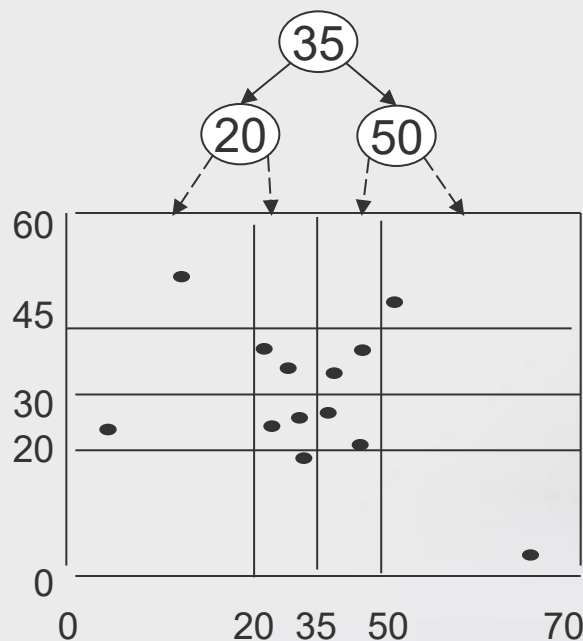
– Grelha Uniforme

Correspondência direta entre posição e array



Grelha Não- Uniforme

A *escala linear* de cada dimensão pode ser representada por uma árvore



Grelhas Dinâmicas

- Se a informação for dinâmica (p.ex. os pontos podem mover-se) ou não estiverem dispersos de forma uniforme pelo espaço, então podem alguns células com muita informação e muitas outras vazias.
 - As grelhas não uniformes tentam dar resposta ao segundo problema, mas uma grelha fixa pode ser insuficiente se existir algum dinamismo ou incerteza em relação à distribuição de pontos.
- Para responder a isto, podem ser usadas **Grelhas Dinâmicas**.



Grelhas Dinâmicas

- Uma forma de implementar grelhas dinâmicas passa pela utilização de uma das seguintes generalizações multidimensionais de árvores binárias (*quadtree* – árvore de quadrantes):
 - **Quadtree de Pontos** – Nesta árvore, as fronteiras são determinadas pelos dados inseridos.
 - **Quadtree PR** – Nesta árvore, as fronteiras são escolhidas de forma predeterminada.



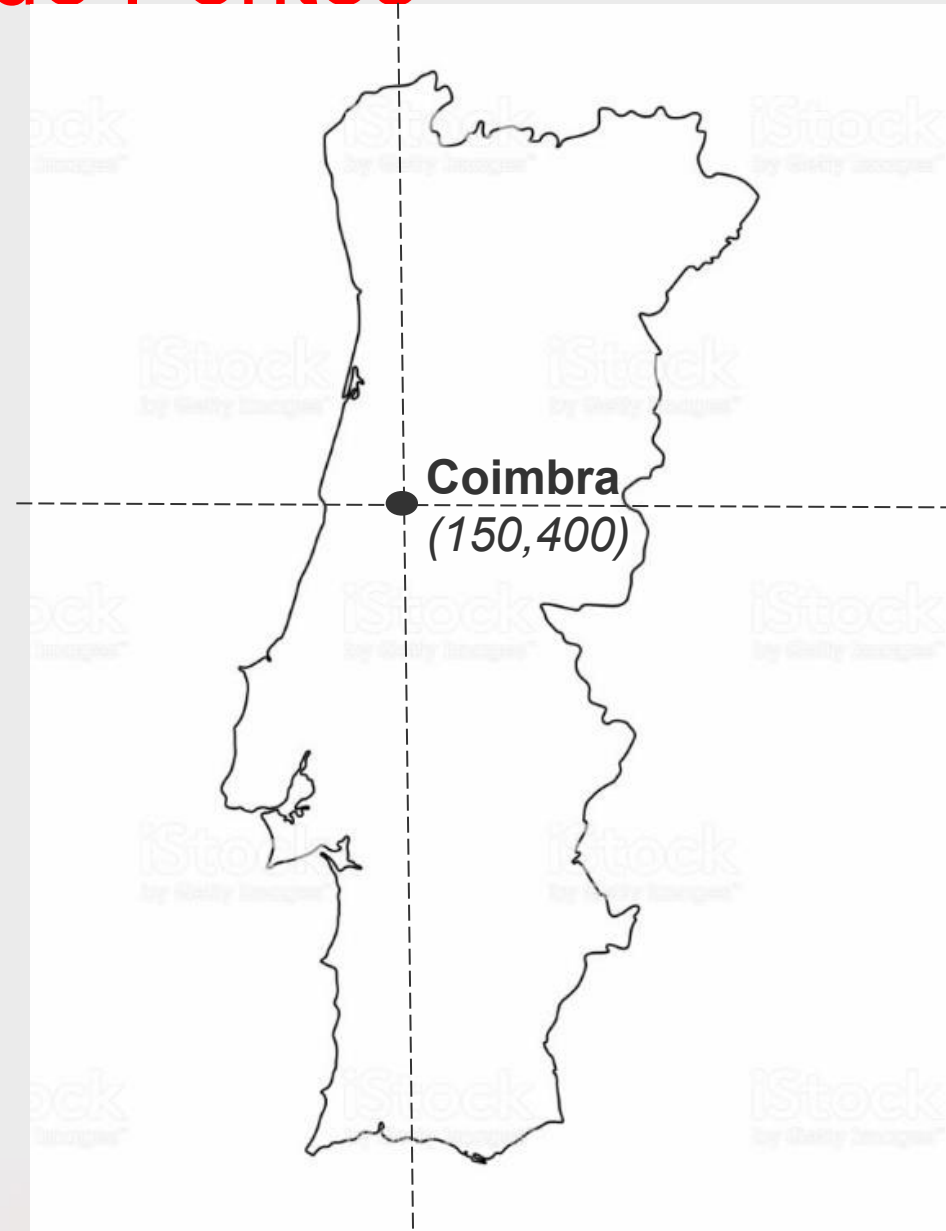
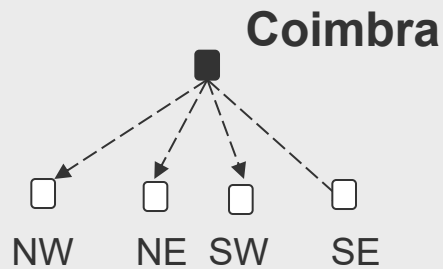
Quadtree de Pontos

- Nesta árvore, **cada ponto inserido vai separar o espaço em vários quadrantes (4 no caso bidimensional)**
 - *A forma da árvore depende da ordem de inserção dos pontos.*



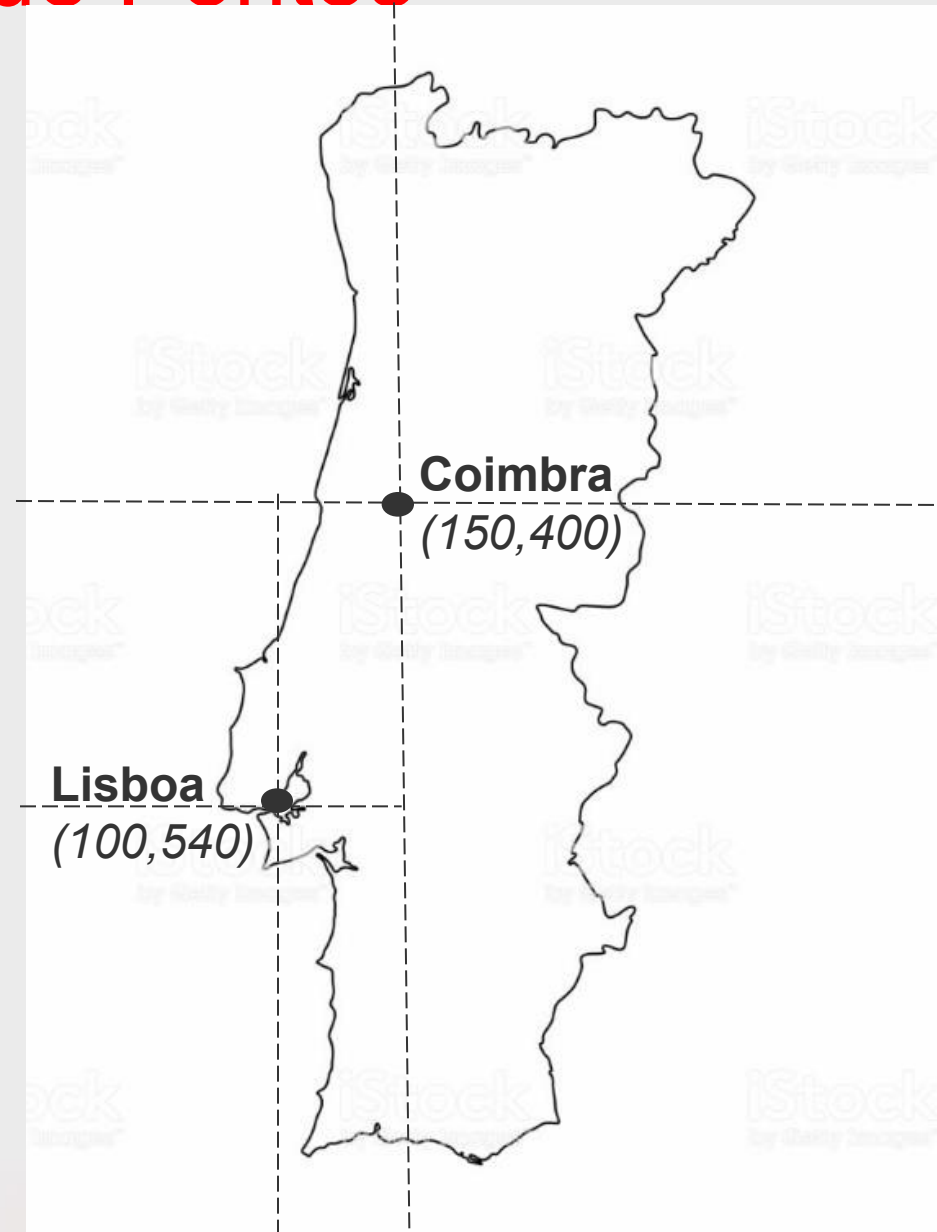
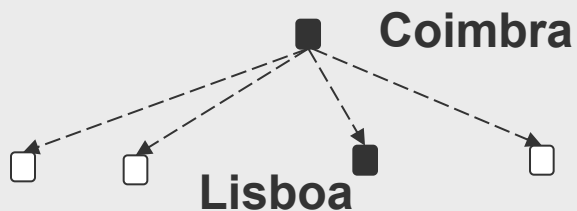
Quadtree de Pontos

1. Inserir Coimbra



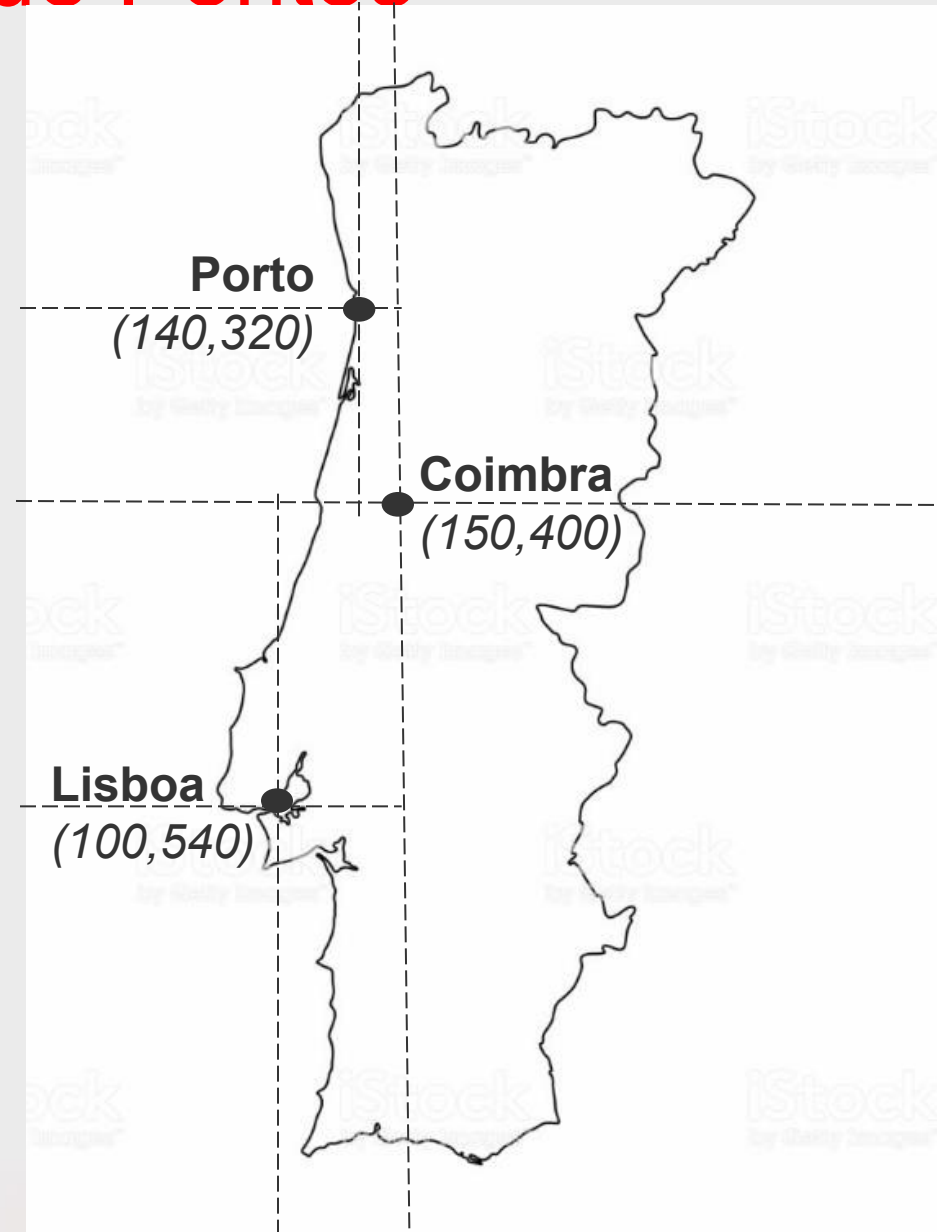
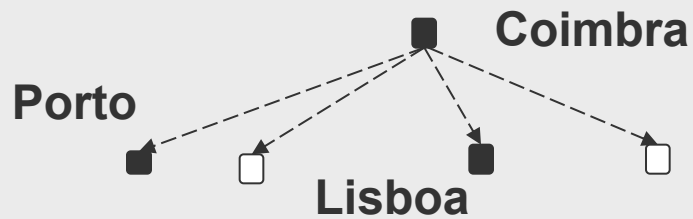
Quadtree de Pontos

1. *Inserir* **Coimbra**
2. *Inserir* **Lisboa**



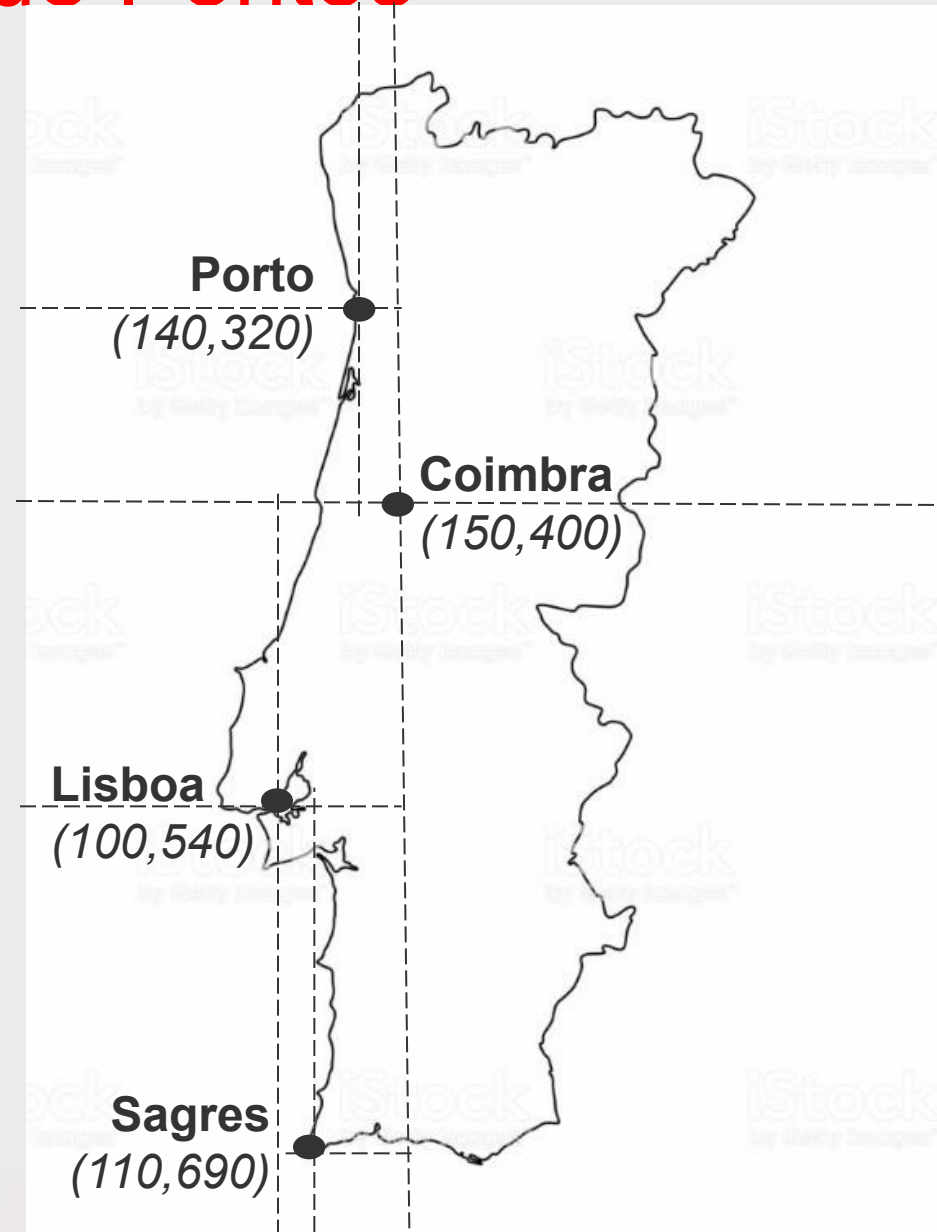
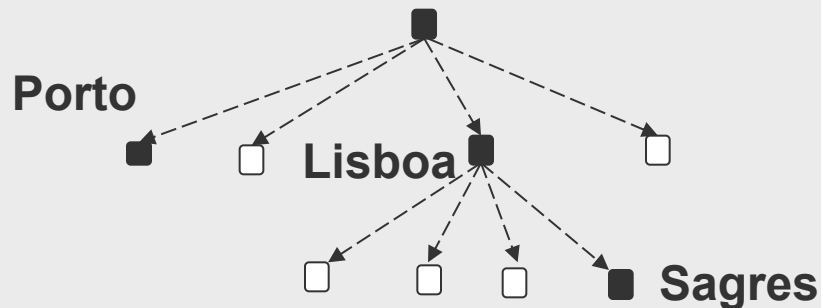
Quadtree de Pontos

1. *Inserir Coimbra*
2. *Inserir Lisboa*
3. *Inserir Porto*

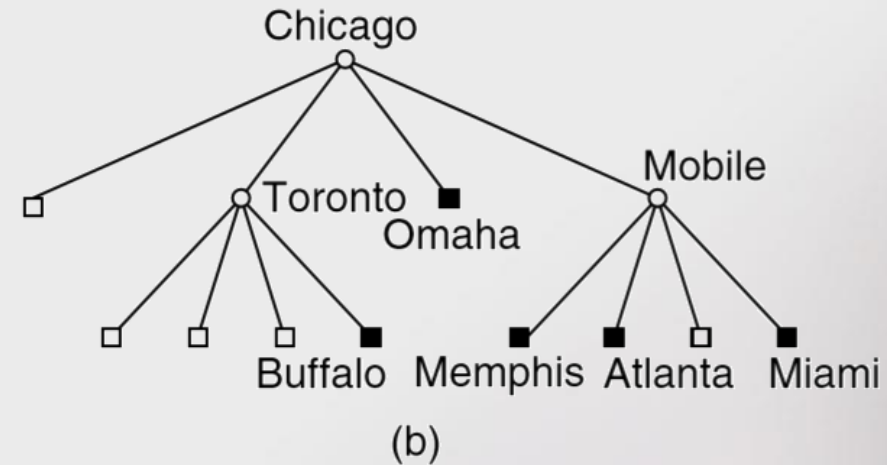
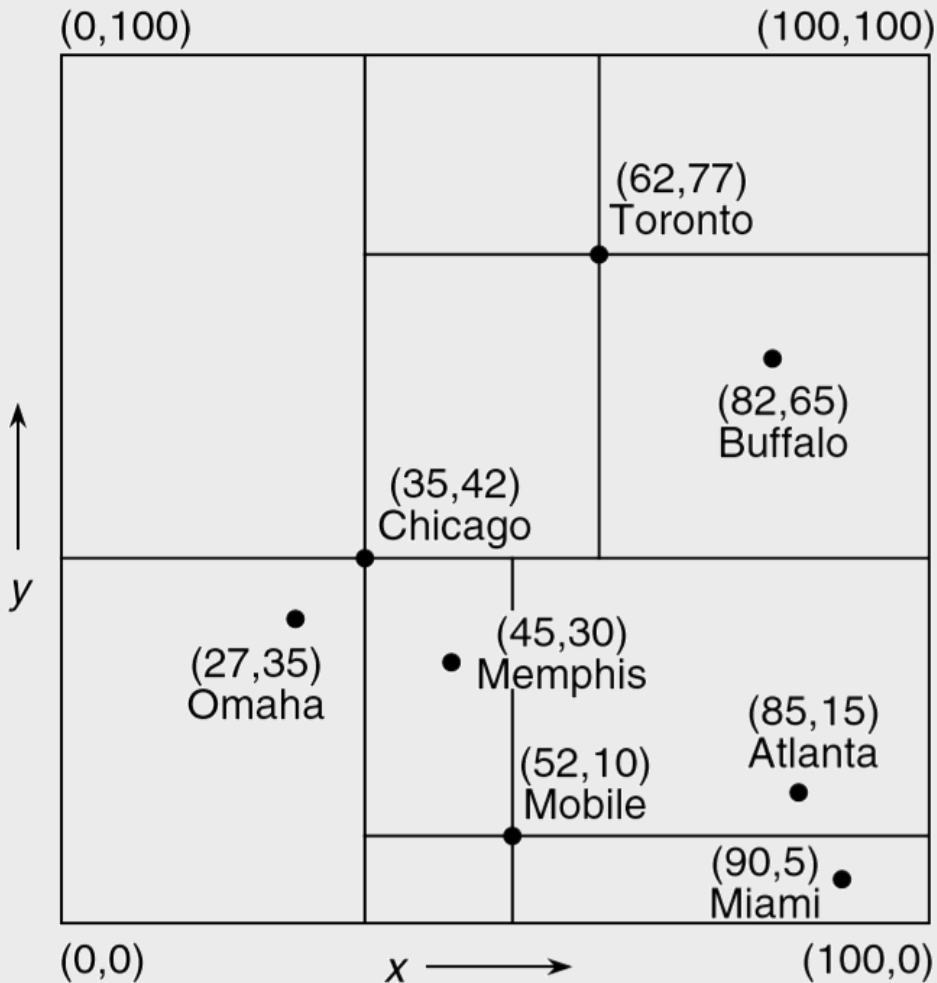


Quadtree de Pontos

1. *Inserir Coimbra*
2. *Inserir Lisboa*
3. *Inserir Porto*
4. *Inserir Sagres*



Quadtree de Pontos



Quadtree PR

- Nesta árvore, o espaço é dividido em 4 quadrantes congruentes (“iguais”), assegurando que em cada célula há no máximo um ponto.
 - A forma da árvore **não** depende da ordem de inserção
 - Os pontos estão nas folhas da árvore



Quadtree PR

1. *Inserir Coimbra*

■ Coimbra

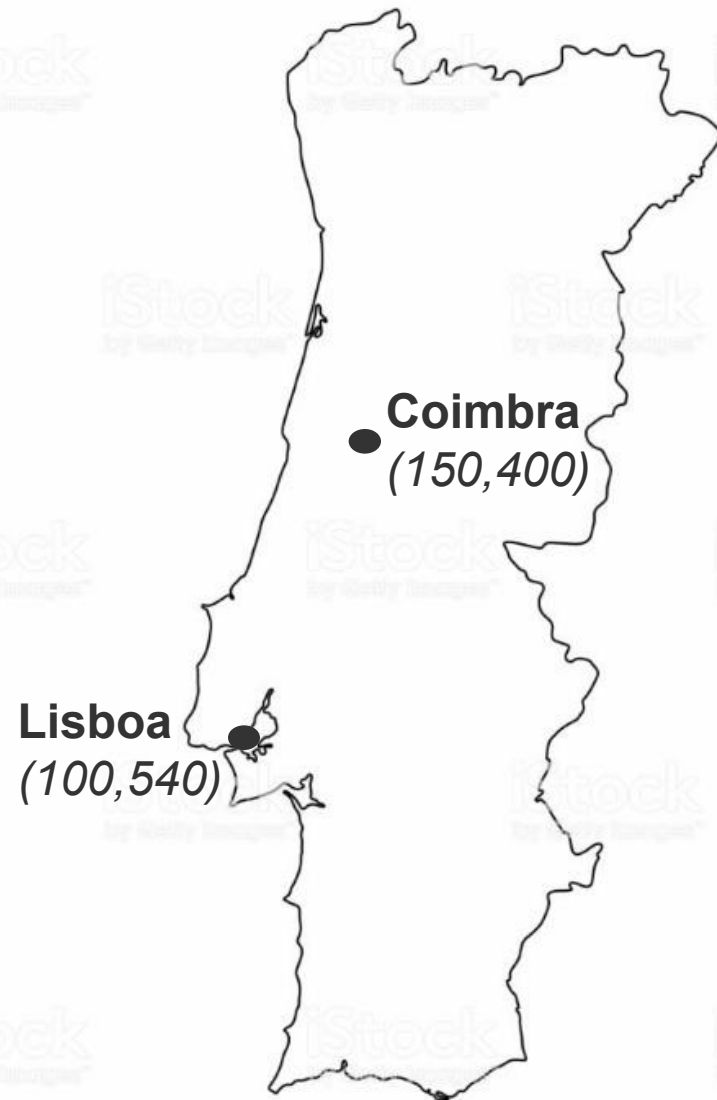


Quadtree PR

1. *Inserir* **Coimbra**
2. *Inserir* **Lisboa**

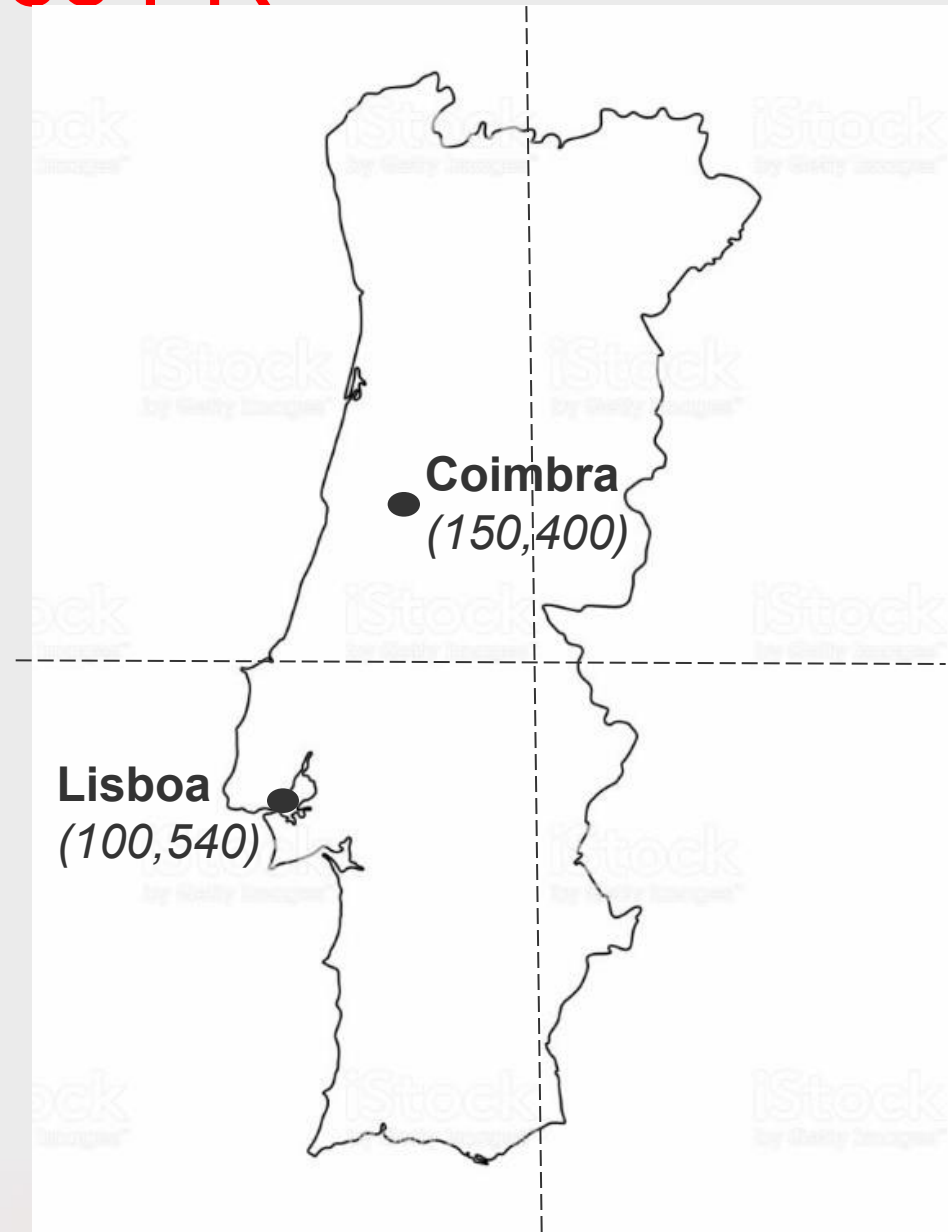
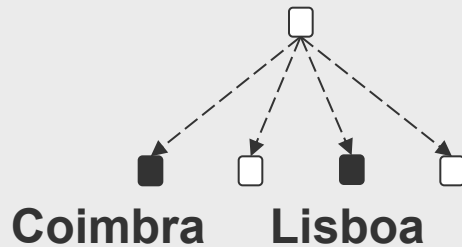
Há apenas uma célula que
contem dois pontos.
Vai ser subdividido para corrigir
essa situação.

■ Coimbra+Lisboa



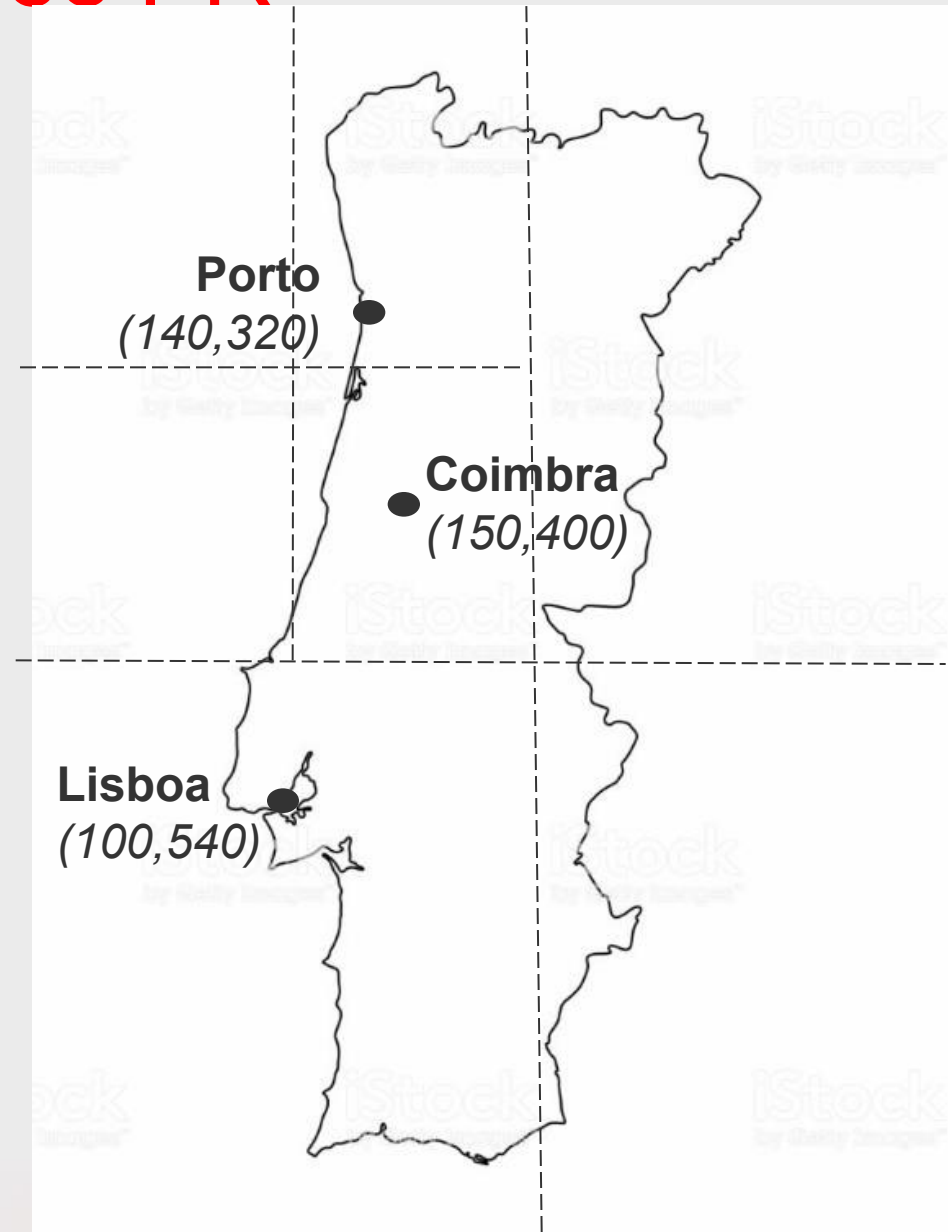
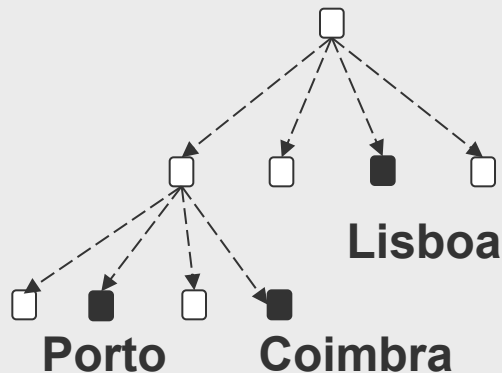
Quadtree PR

1. *Inserir* **Coimbra**
2. *Inserir* **Lisboa**



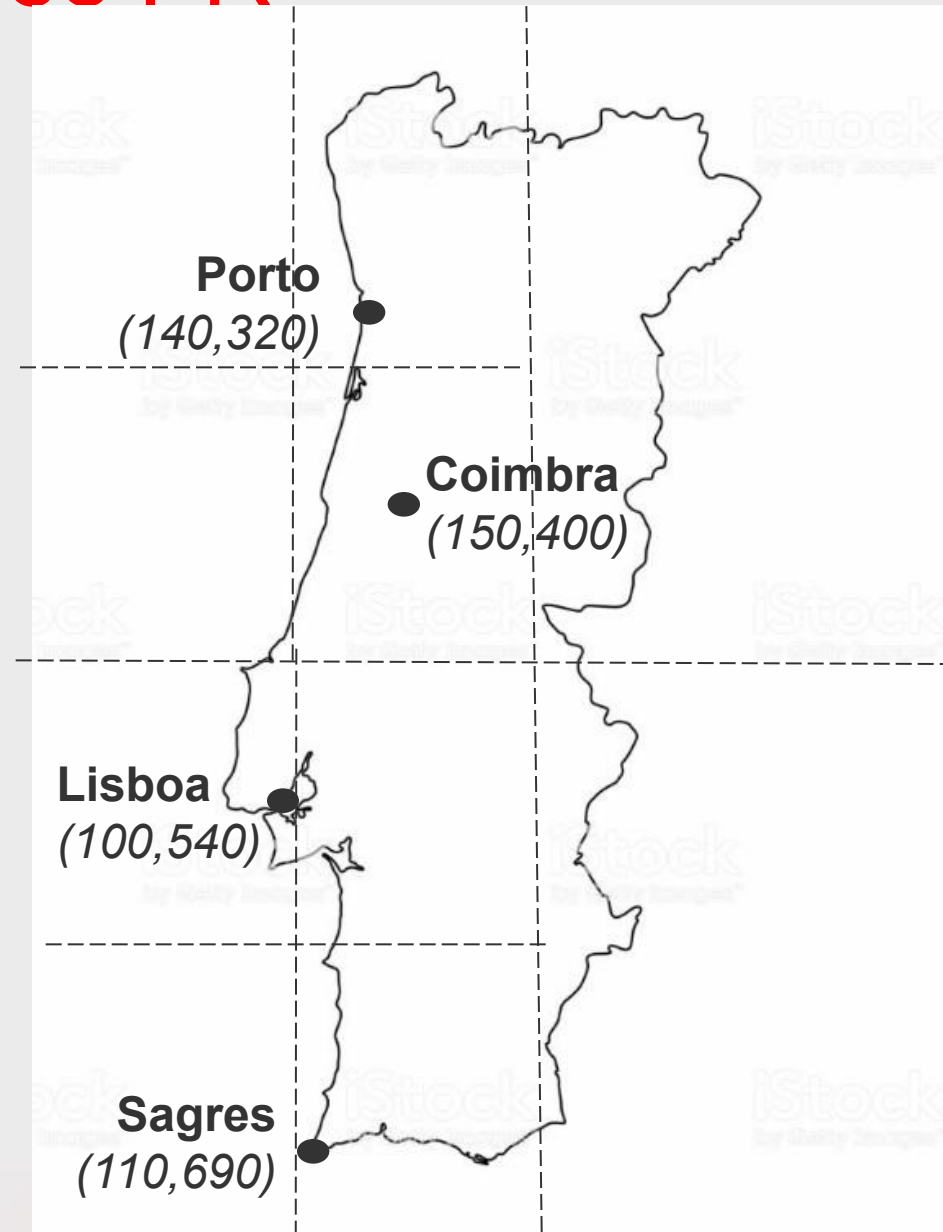
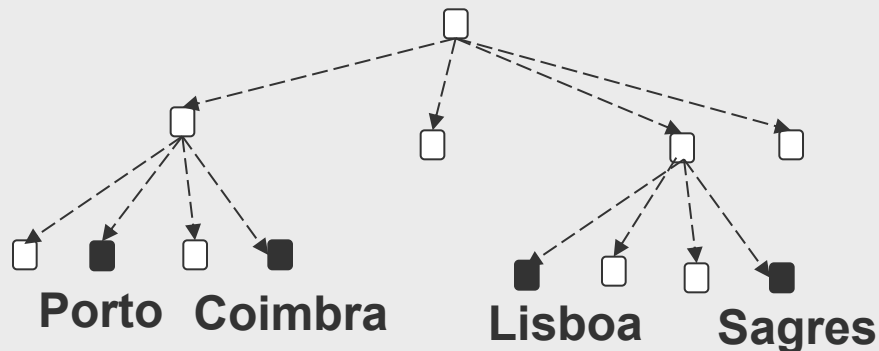
Quadtree PR

1. *Inserir Coimbra*
2. *Inserir Lisboa*
3. *Inserir Porto*



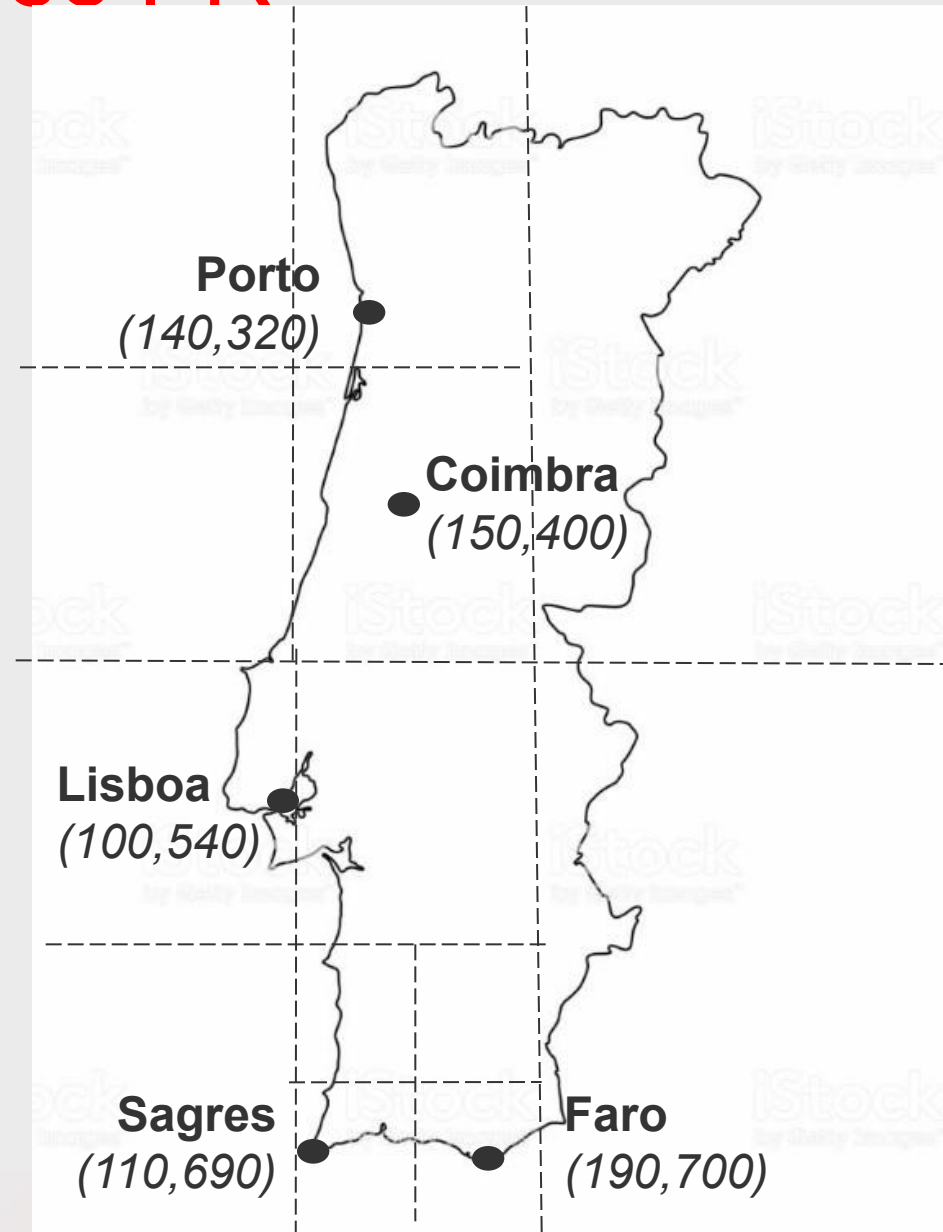
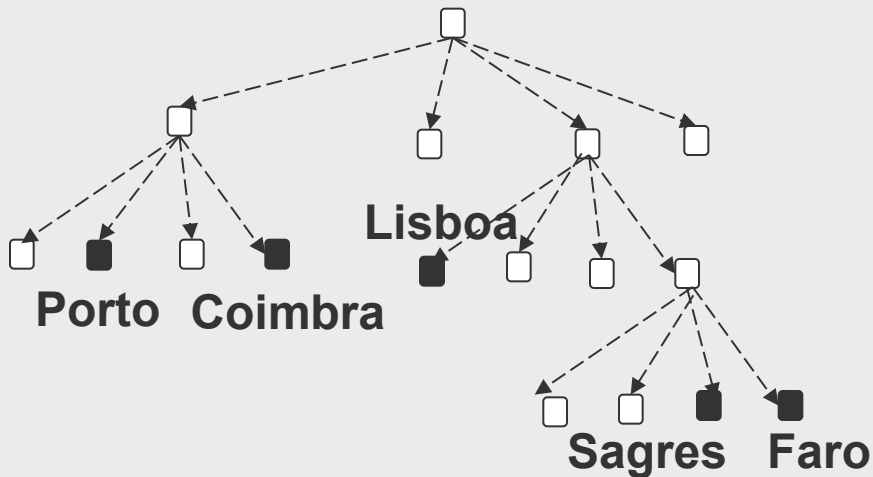
Quadtree PR

1. *Inserir Coimbra*
2. *Inserir Lisboa*
3. *Inserir Porto*
4. *Inserir Sagres*



Quadtree PR

1. *Inserir Coimbra*
2. *Inserir Lisboa*
3. *Inserir Porto*
4. *Inserir Sagres*
5. *Inserir Faro*



Ocupação de espaço

- Numa Quadtree PR o número de células vazias e a profundidade da árvore pode crescer significativamente:
 - A profundidade da árvore aumenta com a menor distância entre dois pontos.
- Para reduzir a dimensão da árvore, podem ser admitidos *buckets* com mais do que um valor.
- Outra forma de ultrapassar este problema é recorrer a **árvores *k-d***.



Árvore k-d

- Uma árvore k-d indexa a informação de acordo com uma dimensão (atributo) distinta em cada nível.
 - *Uma árvore de pontos (x,y,z) compara os pontos de acordo com x no nível 1, 4, 7, 10,..., de acordo com o valor de y nos níveis 2, 5, 8, 11, ... e de acordo com o valor de z nos níveis 3, 6, 9, 12, ...*



Árvore k-d

Exemplo: *Árvore k-d com pessoas (nome, idade, morada)*

1. *Inserir* ("João", 23, "Porto")

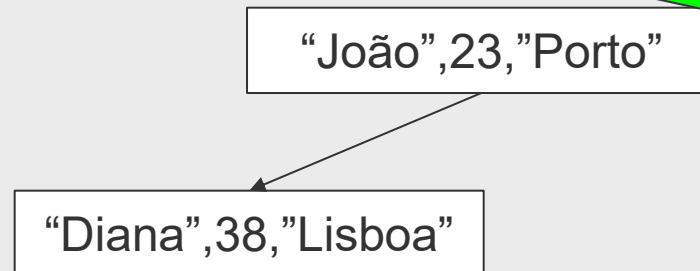
"João", 23, "Porto"



Árvore k-d

Exemplo: *Árvore k-d com pessoas (nome, idade, morada)*

1. **Inserir** ("João", 23, "Porto")
2. **Inserir** ("Diana", 38, "Lisboa")



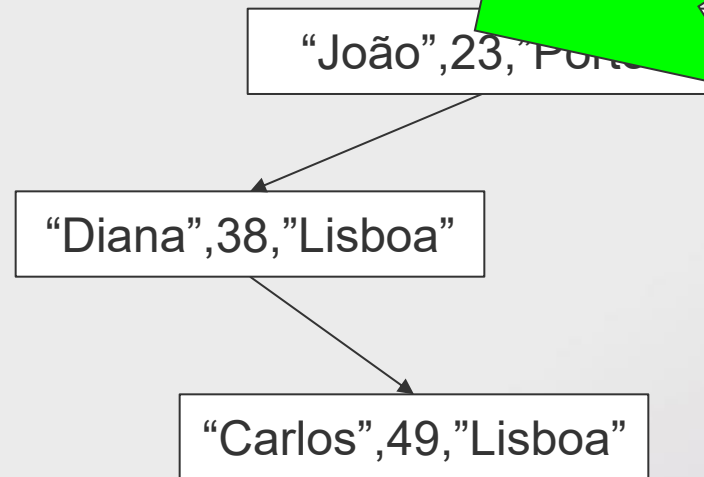
Nível 1:
Comparação por nome
Diana < João



Árvore k-d

Exemplo: Árvore *k-d* com pessoas (*nome, idade, morada*)

1. **Inserir** ("João",23,"Porto")
2. **Inserir** ("Diana",38,"Lisboa")
3. **Inserir** ("Carlos",49,"Lisboa")



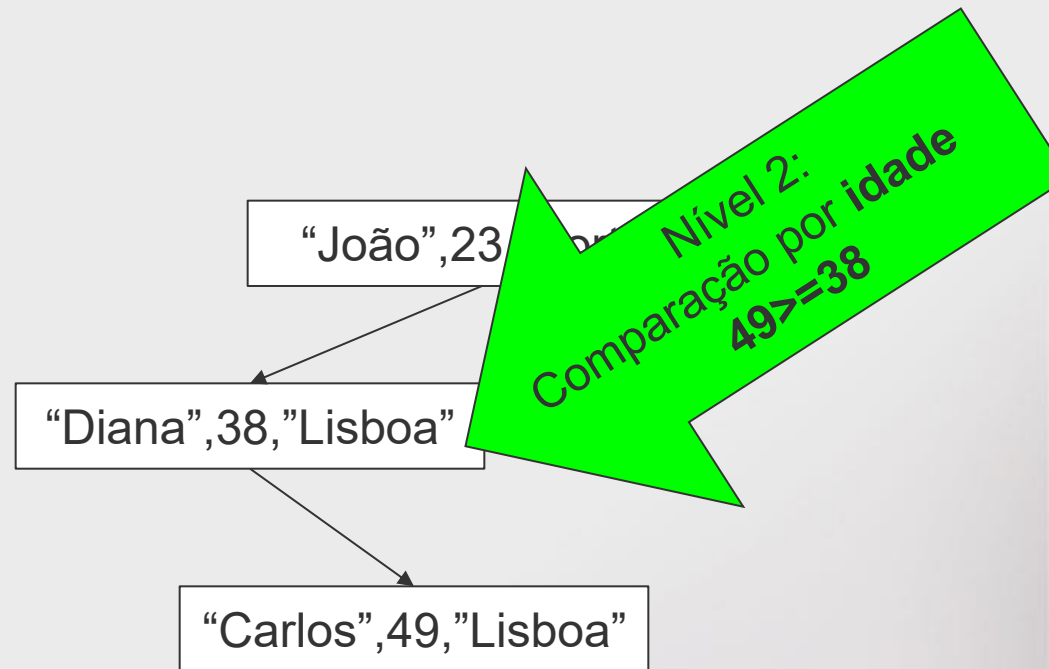
Nível 1:
Comparação por nome
Carlos < João



Árvore k-d

Exemplo: Árvore *k-d* com pessoas (*nome, idade, morada*)

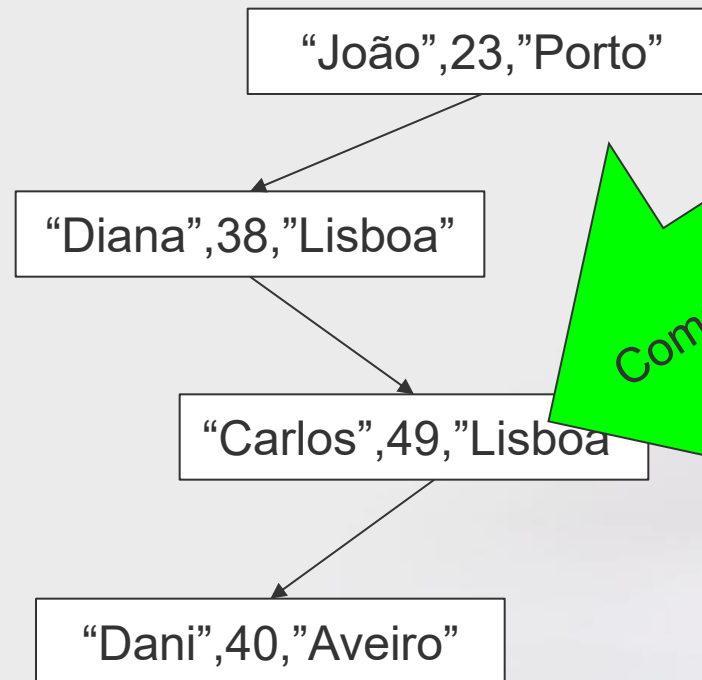
1. **Inserir** ("João",23,"Porto")
2. **Inserir** ("Diana",38,"Lisboa")
3. **Inserir** ("Carlos",49,"Lisboa")



Árvore k-d

Exemplo: *Árvore k-d com pessoas (nome, idade, morada)*

1. **Inserir** ("João", 23, "Porto")
2. **Inserir** ("Diana", 38, "Lisboa")
3. **Inserir** ("Carlos", 49, "Lisboa")
4. **Inserir** ("Dani", 40, "Aveiro")



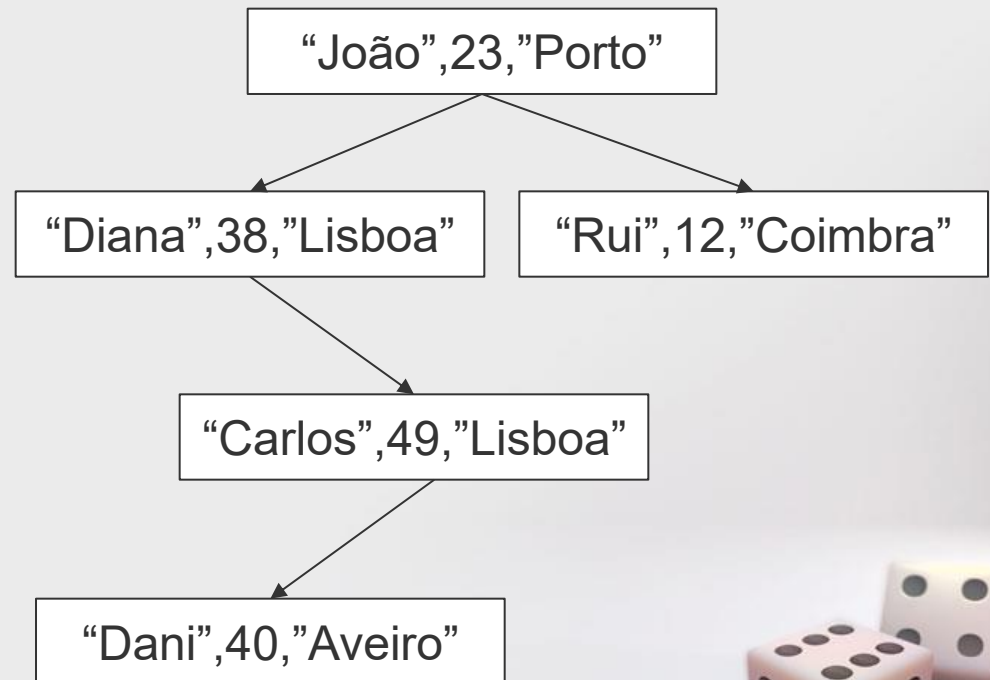
Nível 3:
Comparação por morada
Aveiro < Lisboa



Árvore k-d

Exemplo: *Árvore k-d com pessoas (nome, idade, morada)*

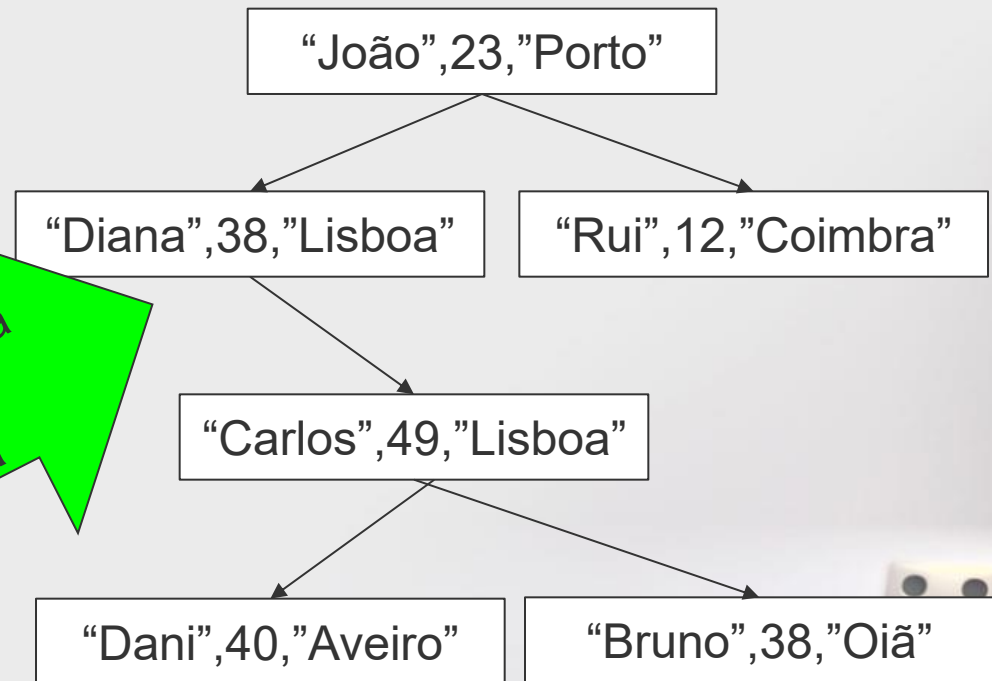
1. **Inserir** ("João", 23, "Porto")
2. **Inserir** ("Diana", 38, "Lisboa")
3. **Inserir** ("Carlos", 49, "Lisboa")
4. **Inserir** ("Dani", 40, "Aveiro")
5. **Inserir** ("Rui", 12, "Coimbra")



Árvore k-d

Exemplo: *Árvore k-d com pessoas (nome, idade, morada)*

1. **Inserir** ("João",23,"Porto")
2. **Inserir** ("Diana",38,"Lisboa")
3. **Inserir** ("Carlos",49,"Lisboa")
4. **Inserir** ("Dani",40,"Aveiro")
5. **Inserir** ("Rui",12,"Coimbra")
6. **Inserir** ("Bruno",38,"Oiã")



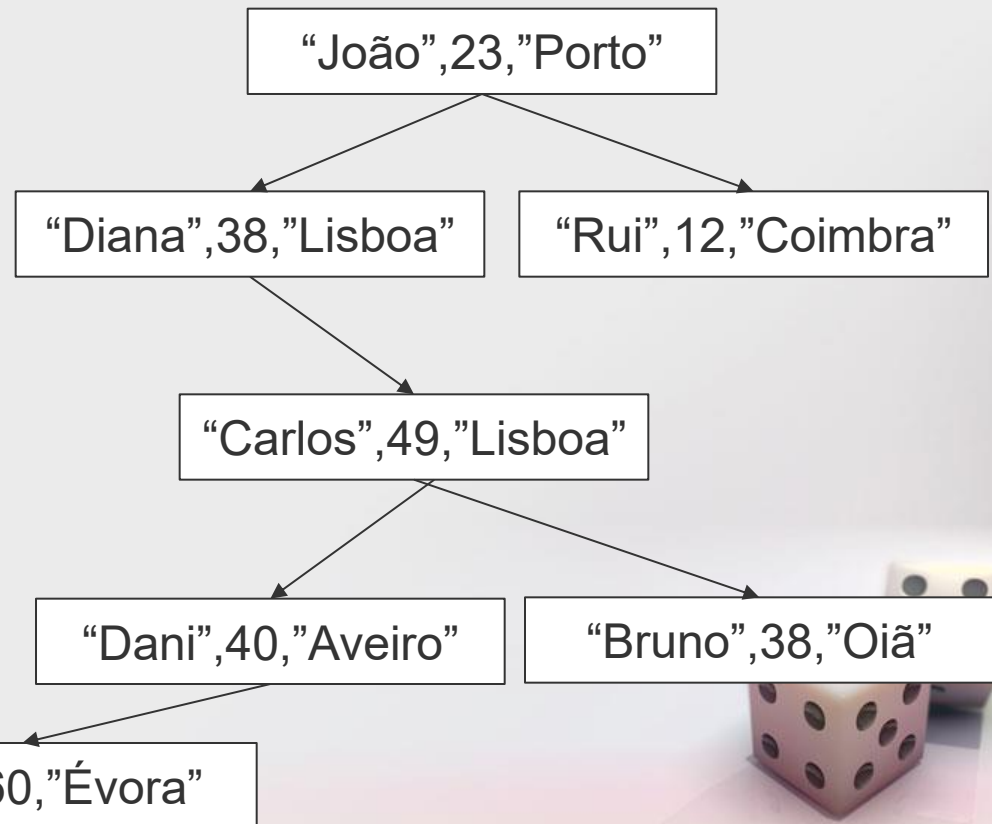
Podem existir valores iguais numa determinada dimensão!
38 >= 38 vamos para direita



Árvore k-d

Exemplo: *Árvore k-d com pessoas (nome, idade, morada)*

1. **Inserir** ("João", 23, "Porto")
2. **Inserir** ("Diana", 38, "Lisboa")
3. **Inserir** ("Carlos", 49, "Lisboa")
4. **Inserir** ("Dani", 40, "Aveiro")
5. **Inserir** ("Rui", 12, "Coimbra")
6. **Inserir** ("Bruno", 38, "Oiã")
7. **Inserir** ("Alex", 60, "Évora")



Árvores k-d

- As árvores k-d podem ser usadas independentemente, ou então serem usadas para implementar a decomposição do espaço das quadrees



Árvore PR k-d

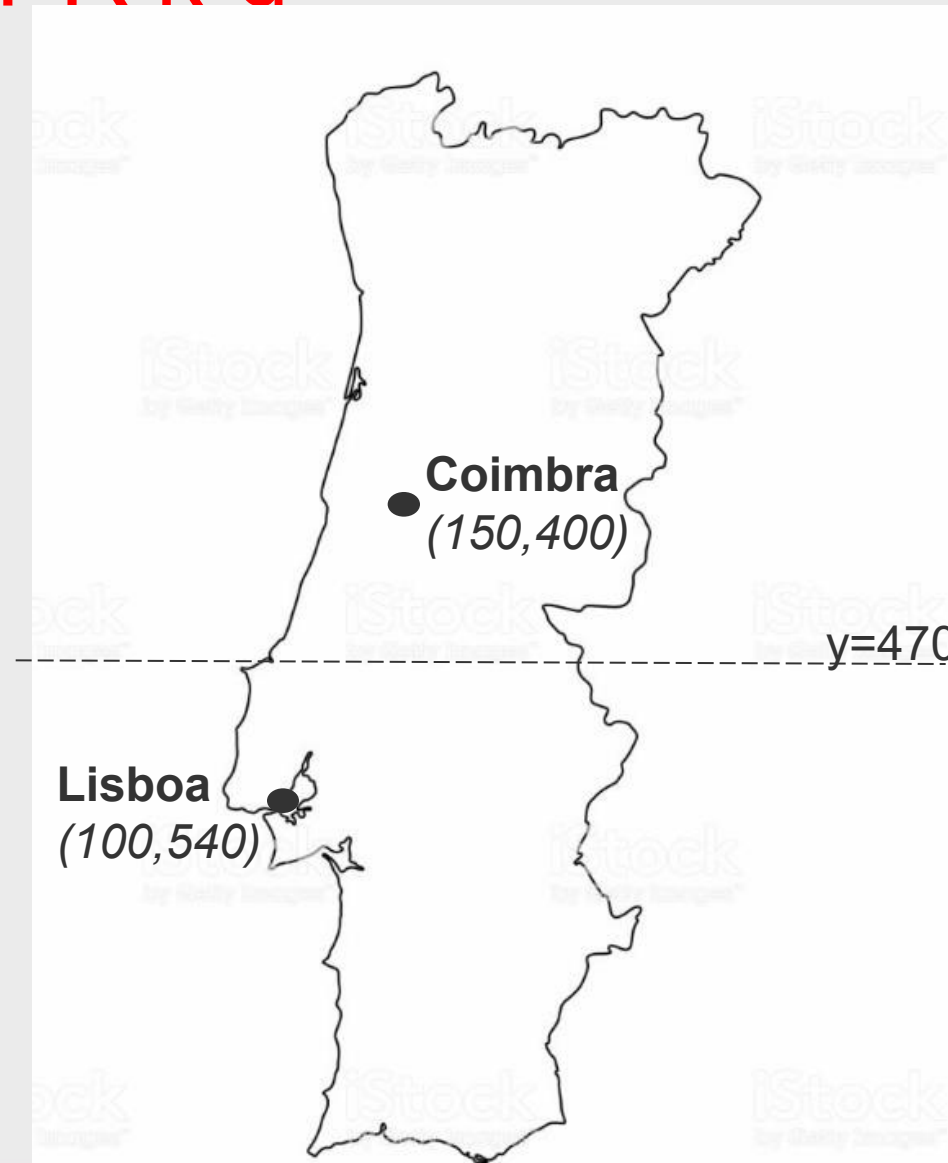
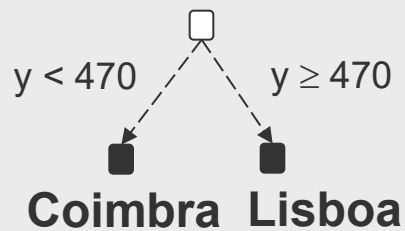
1. *Inserir* **Coimbra**

■
Coimbra



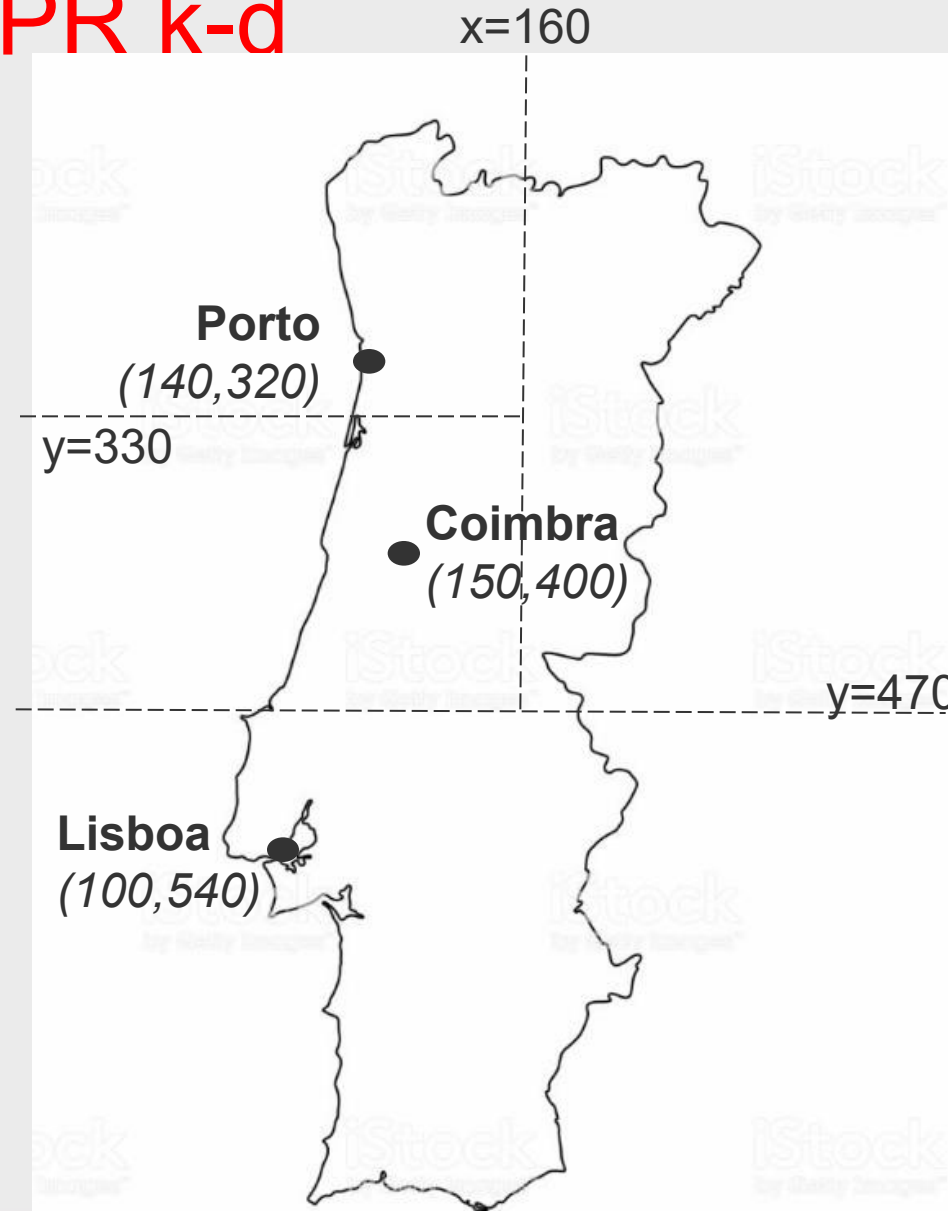
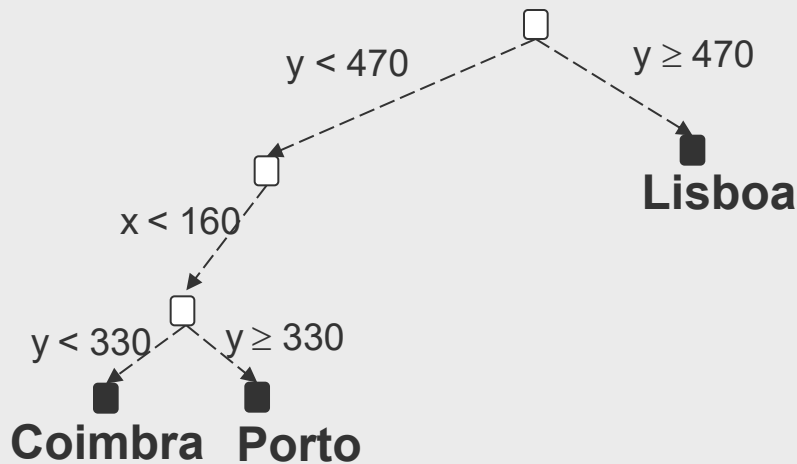
Árvore PR k-d

1. *Inserir Coimbra*
2. *Inserir Lisboa*



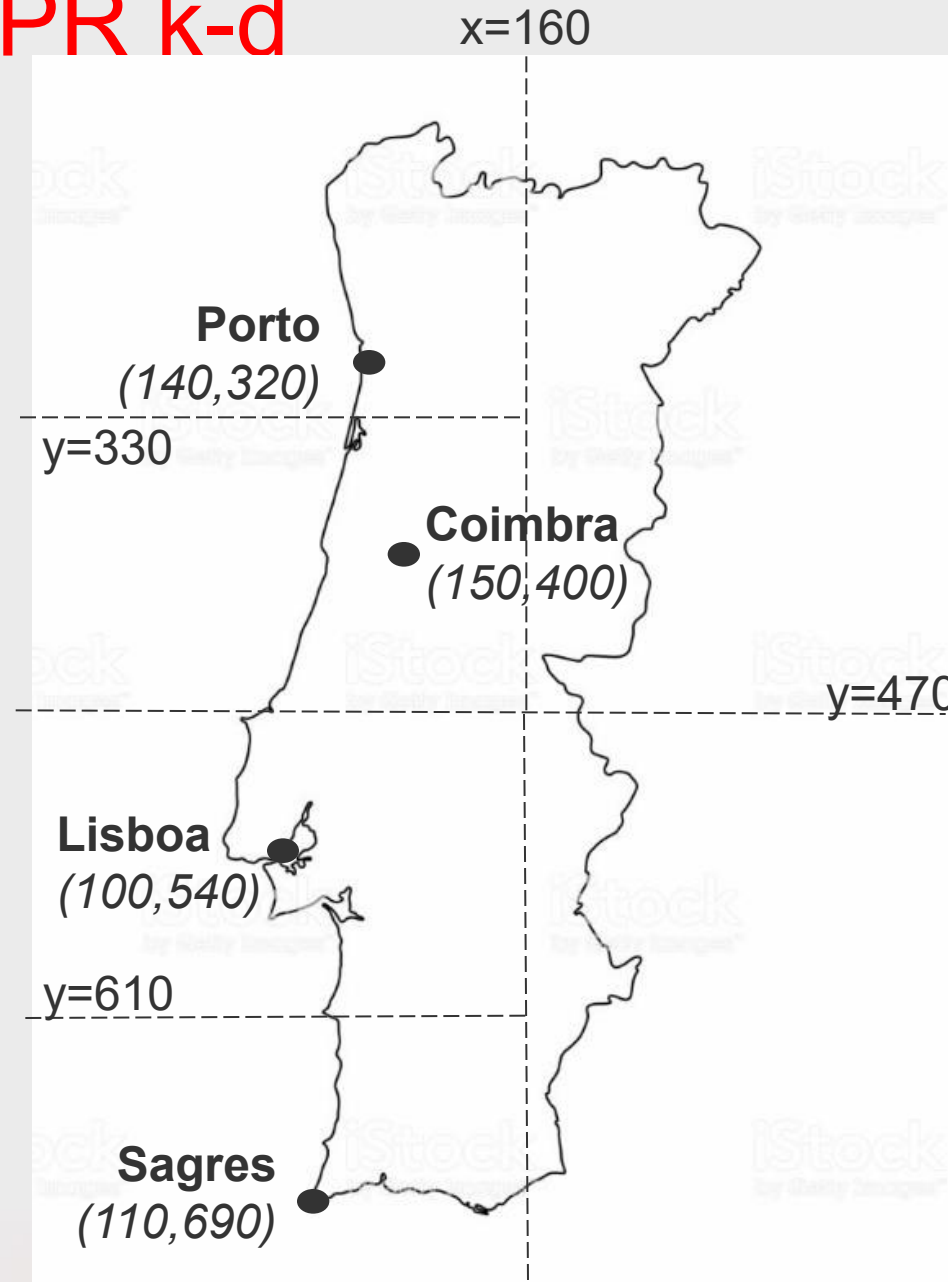
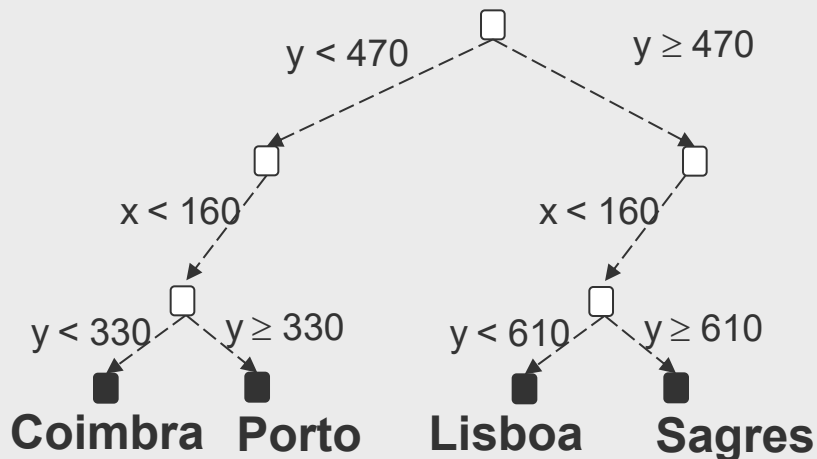
Árvore PR k-d

1. *Inserir Coimbra*
2. *Inserir Lisboa*
3. *Inserir Porto*



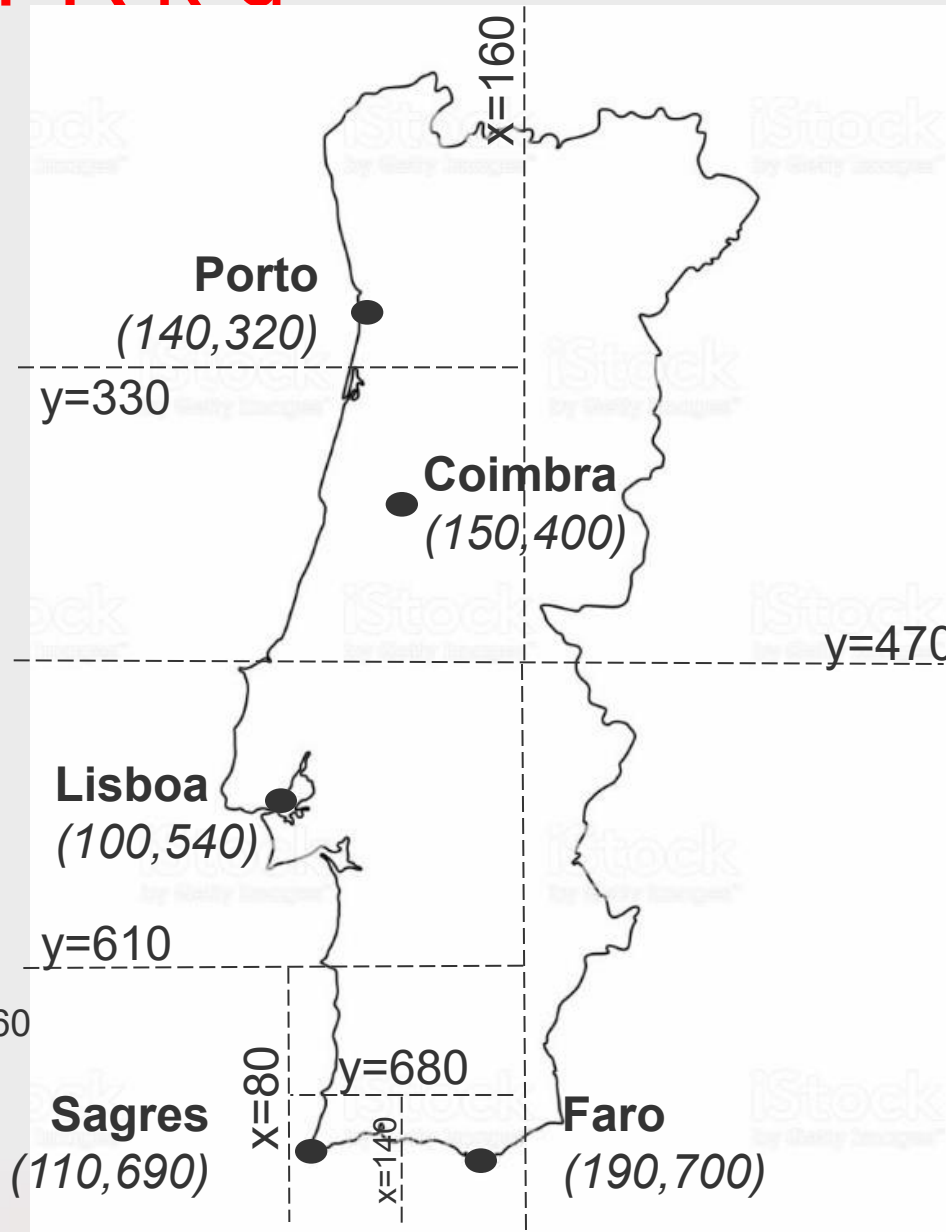
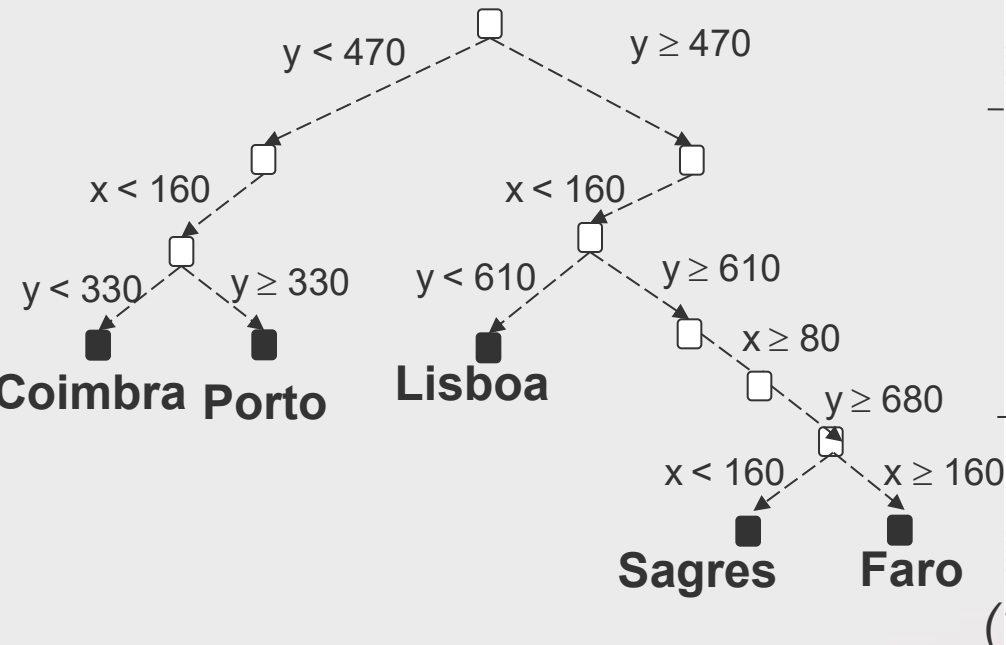
Árvore PR k-d

1. Inserir **Coimbra**
2. Inserir **Lisboa**
3. Inserir **Porto**
4. Inserir **Sagres**



Árvore PR k-d

1. *Inserir* **Coimbra**
2. *Inserir* **Lisboa**
3. *Inserir* **Porto**
4. *Inserir* **Sagres**
5. *Inserir* **Faro**



Pesquisas

- Grelhas fixas, quadrtrees e árvores k-d permitem realizar de forma eficiente pesquisas relacionadas com os *pontos* armazenados como por exemplo:
 - *Quais as cidades a menos de 150 km do Porto?*
 - *Qual é a cidade mais próxima de Sagres?*



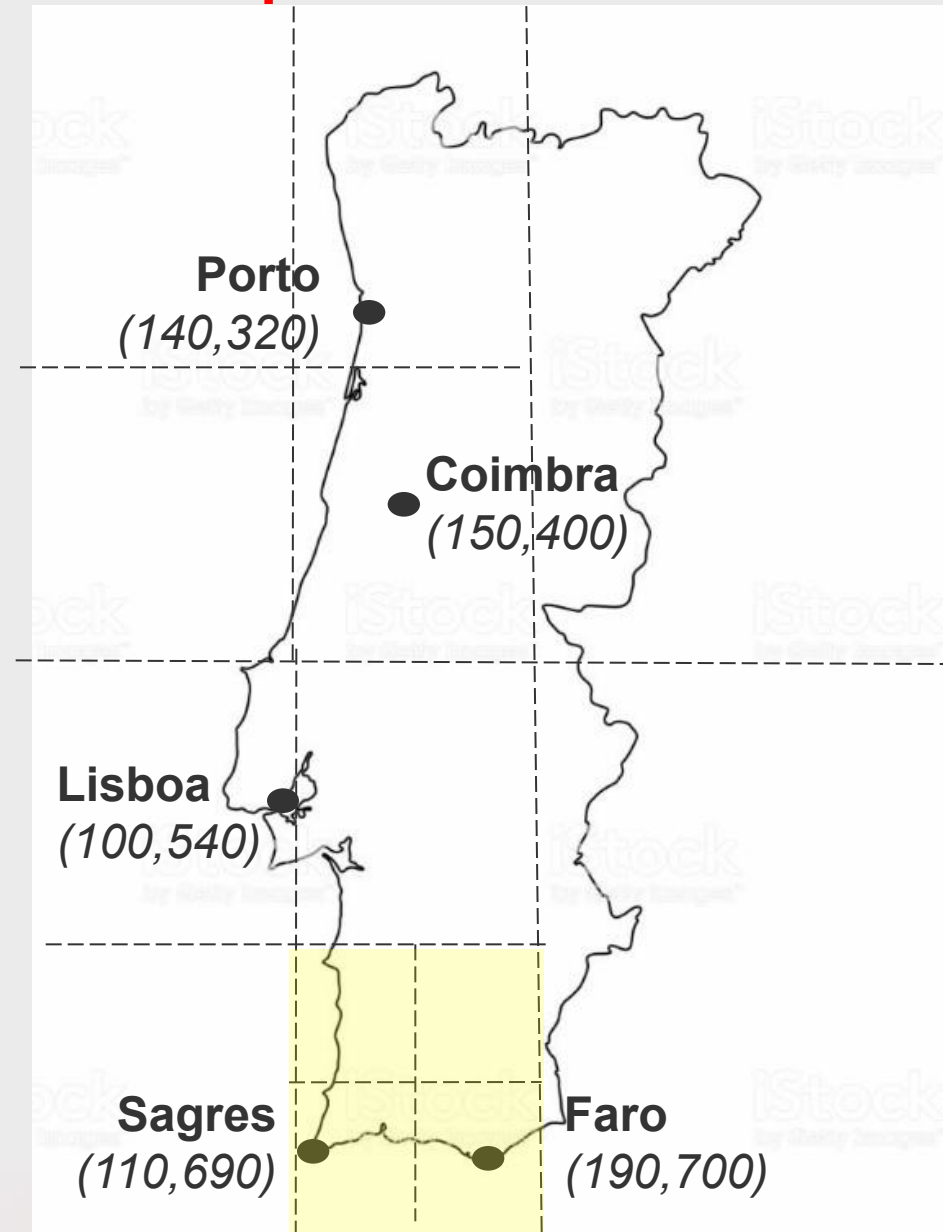
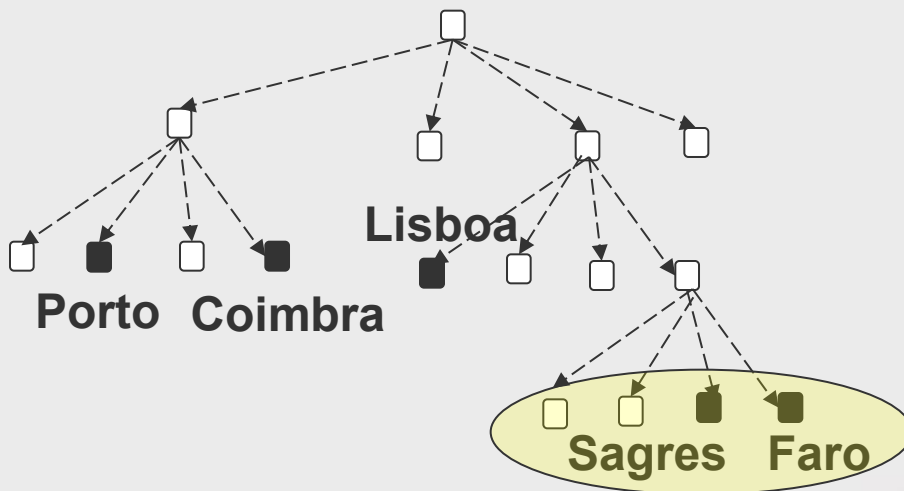
A map of Portugal with a dashed grid. A yellow circle highlights the northern region. The following table lists the cities and their coordinates as shown on the map:

City	Coordinates
Porto	(140, 320)
Coimbra	(150, 400)
Lisboa	(100, 540)
Sagres	(110, 690)
Faro	(190, 700)

-
- The diagram illustrates a hierarchical tree structure. The root node is a white square. It has four children: a white square, a white square, a white square, and a white square. The first child is a white square. The second child is a white square. The third child is a white square. The fourth child is a white square. The first child has four children: a white square, a dark grey square, a white square, and a dark grey square. The second child has one child: a white square. The third child has three children: a dark grey square, a white square, and a white square. The fourth child has four children: a white square, a white square, a dark grey square, and a dark grey square. The nodes are labeled 'Porto', 'Coimbra', 'Lisboa', 'Sagres', and 'Faro'.

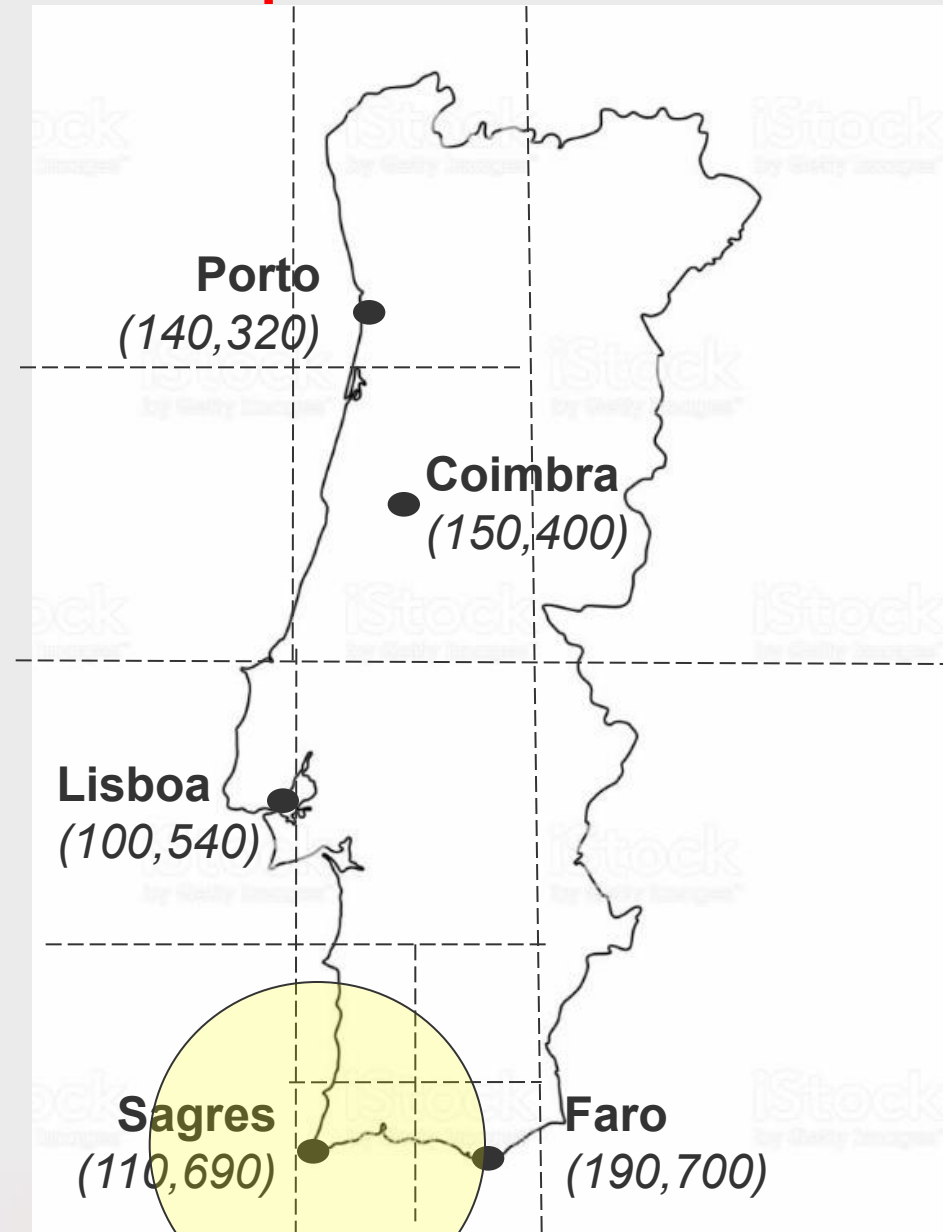
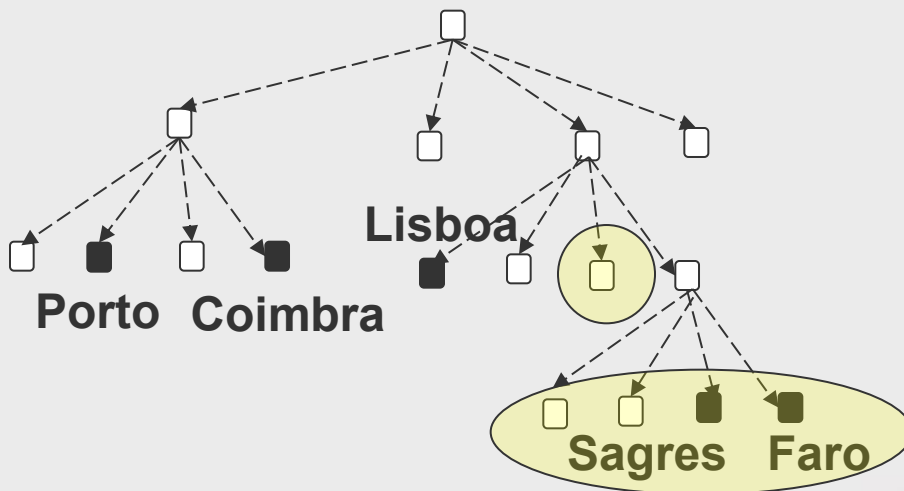
Exemplo de Pesquisa

- Qual a cidade mais próxima de Sagres?
 - Pesquisa-se a cidade mais próxima dentro de cada quadrante*
 - limita-se a pesquisa em função do melhor candidato apurado.*



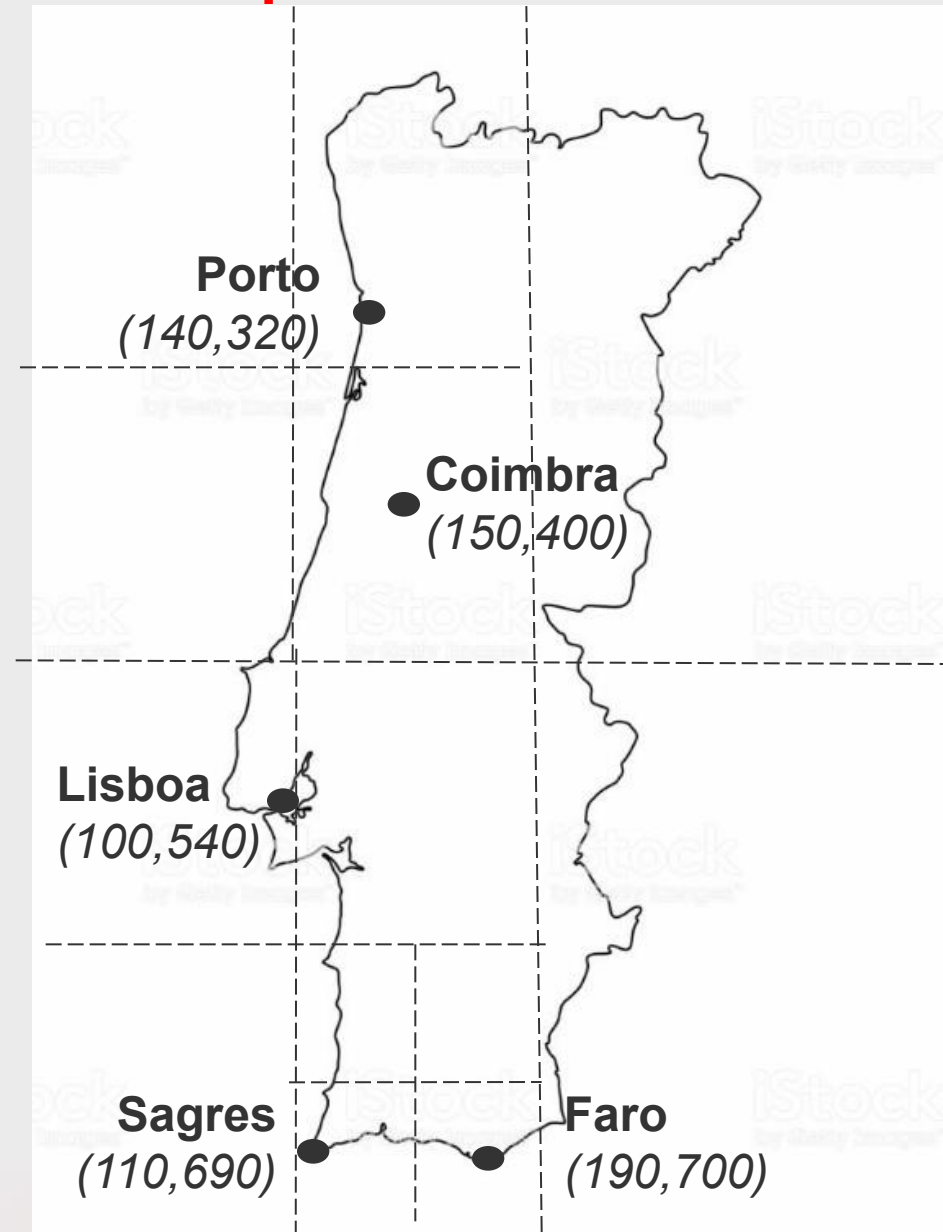
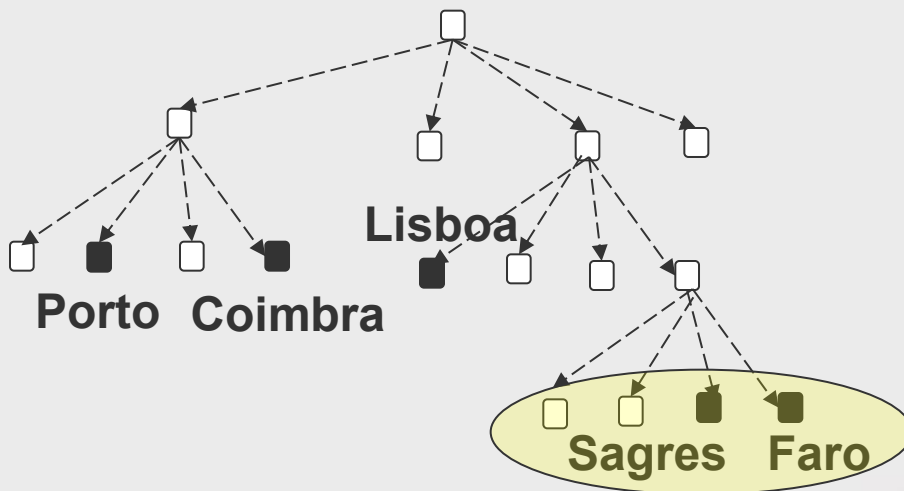
Exemplo de Pesquisa

- Qual a cidade mais próxima de Sagres?
- Pesquisa-se a cidade mais próxima dentro de cada quadrante*
- limita-se a pesquisa em função do melhor candidato apurado.*



Exemplo de Pesquisa

- Qual a cidade mais próxima de Sagres?
 - Pesquisa-se a cidade mais próxima dentro de cada quadrante*
 - limita-se a pesquisa em função do melhor candidato apurado.*



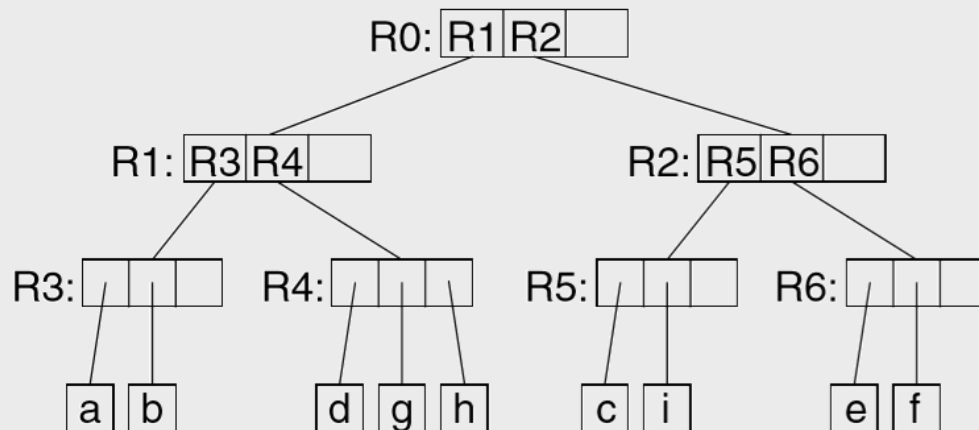
R-Tree

- A R-Tree é um exemplo de uma árvore que utiliza uma hierquia de objetos para decompor o espaço.
 - *Utilizada em motores de bases de dados*
 - *Ao contrário das abordagens anteriores, considera a **área** ocupada pelos pontos*
- Cada nodo contém uma descrição de uma região que contém os pontos contidos na sua sub-árvore.
 - *Tipicamente a região é um hiperrectângulo (retângulo multidimensional)*

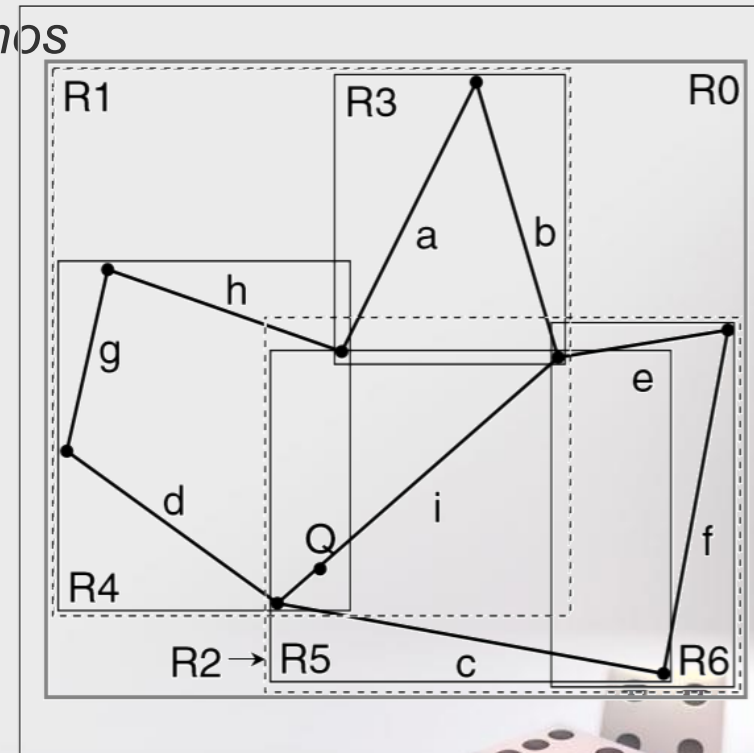


R-Tree

- As R-Tree seguem uma estrutura análoga à das B-Tree:
 - Regiões são índices nos nodos internos
 - Dados estão nas folhas



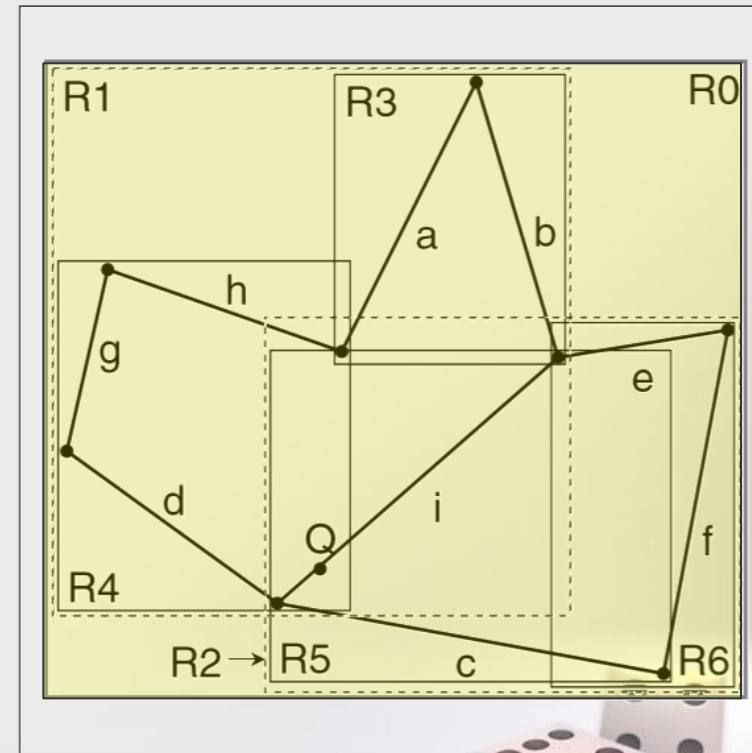
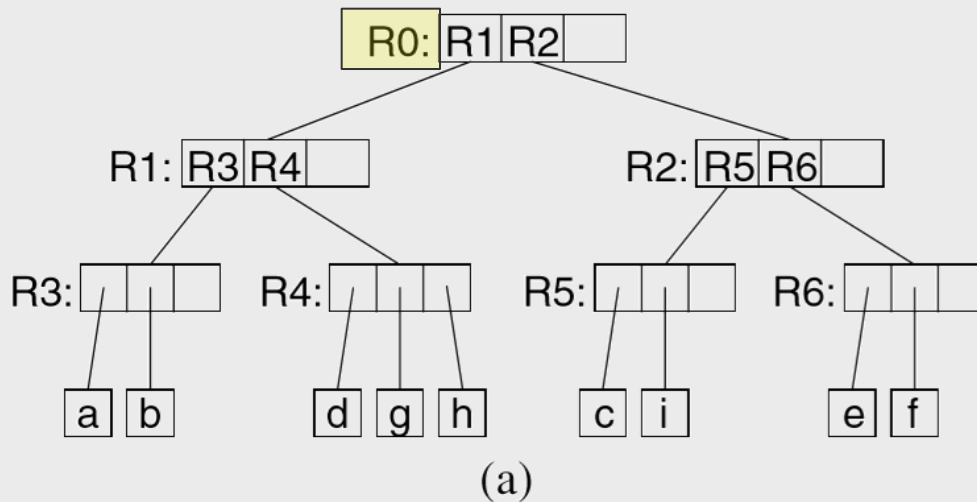
(a)



(b)

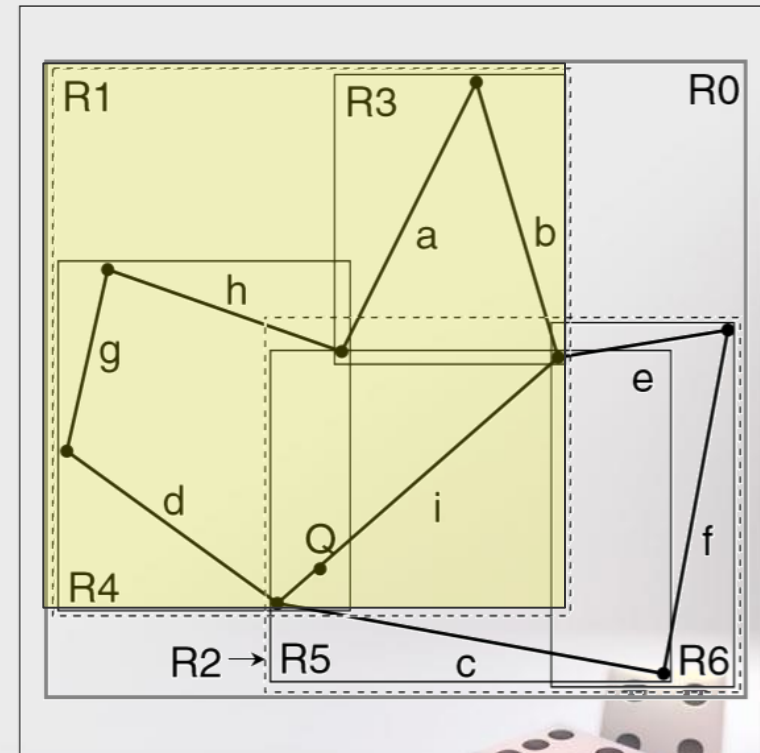
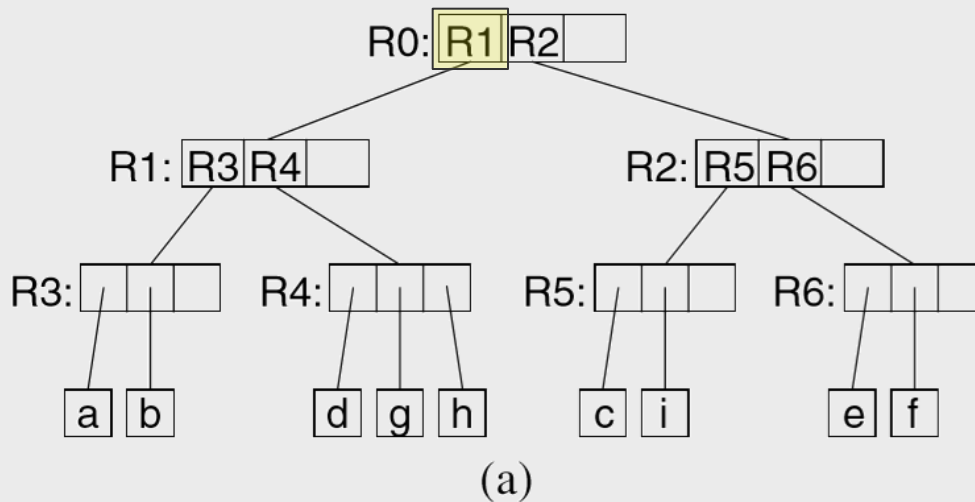
R-Tree

- As R-Tree seguem uma estrutura análoga à das B-Tree



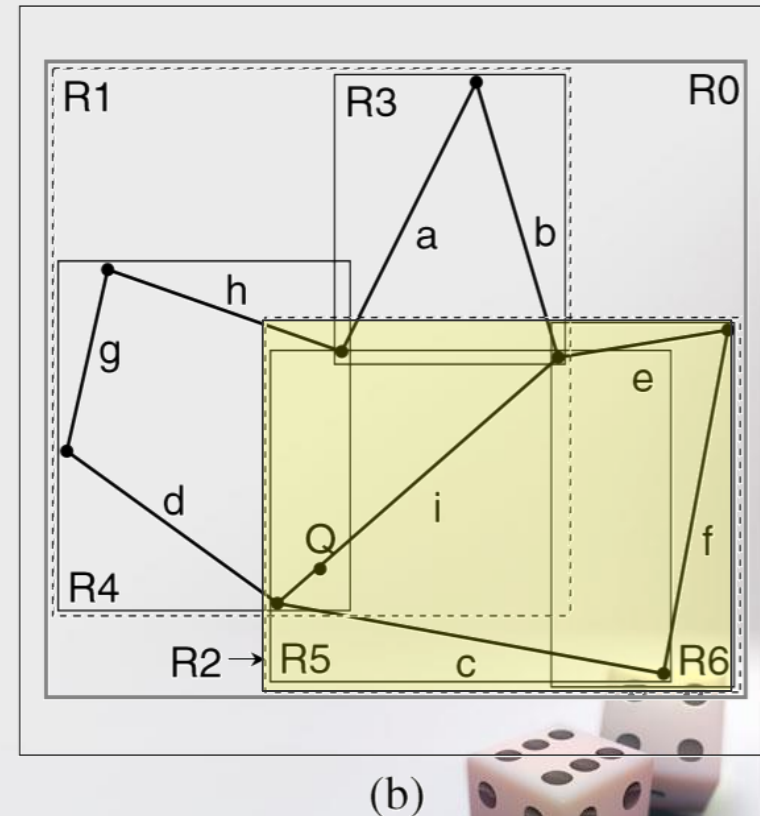
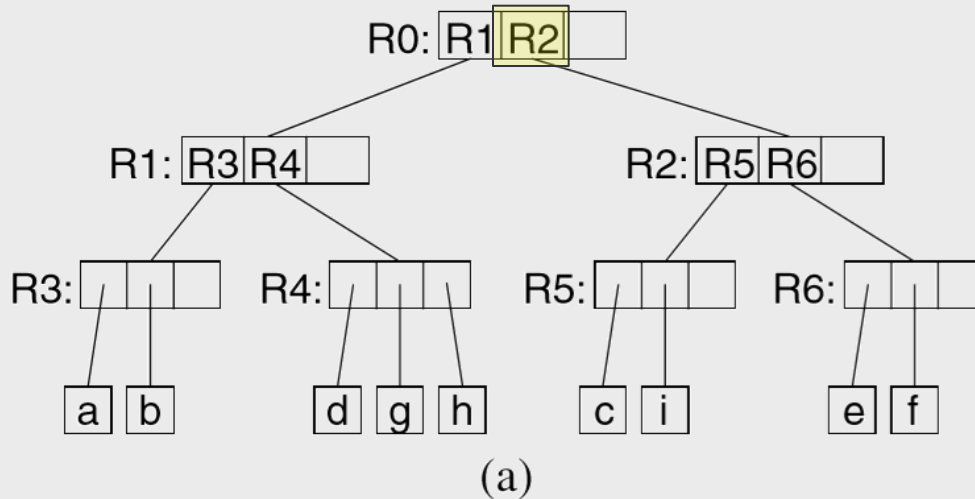
R-Tree

- As R-Tree seguem uma estrutura análoga à das B-Tree



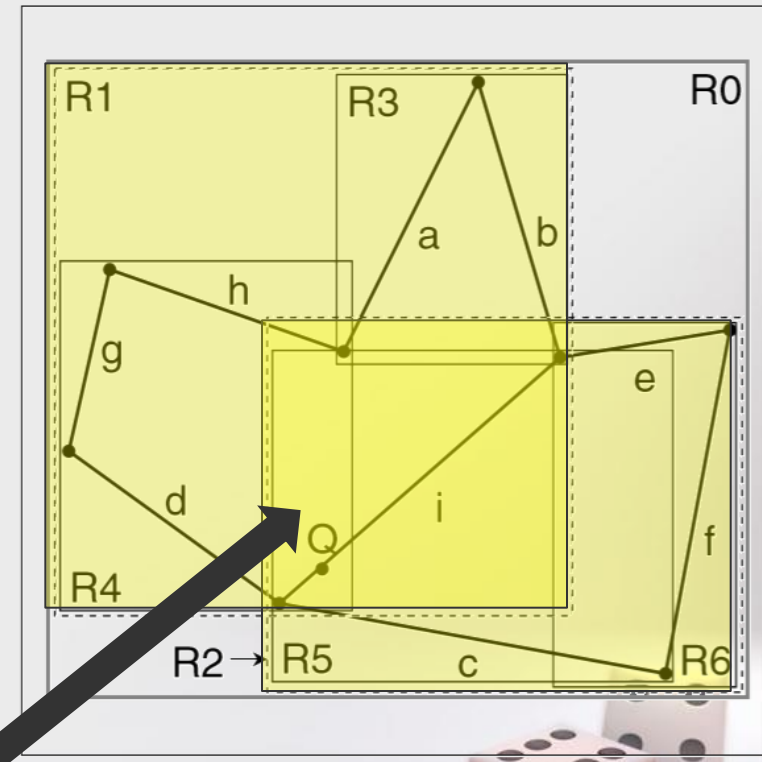
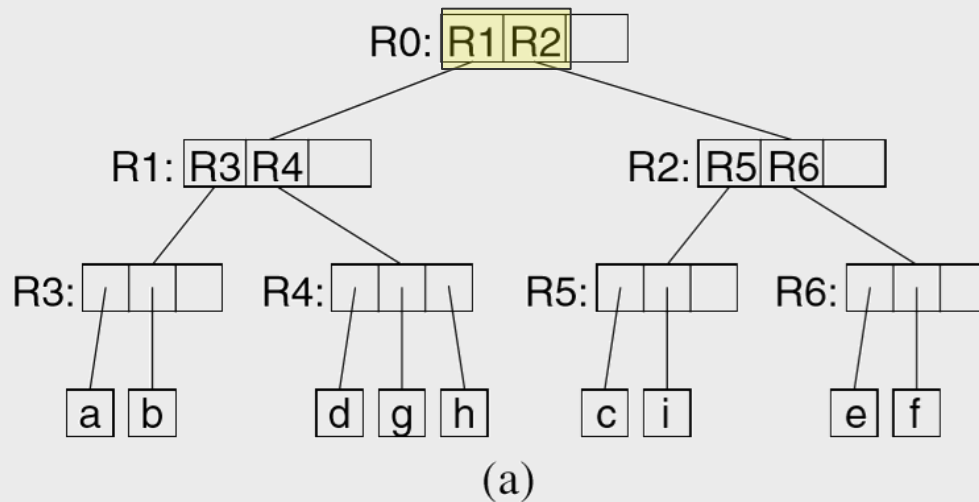
R-Tree

- As R-Tree seguem uma estrutura análoga à das B-Tree



R-Tree

- As R-Tree seguem uma estrutura análoga à das B-Tree



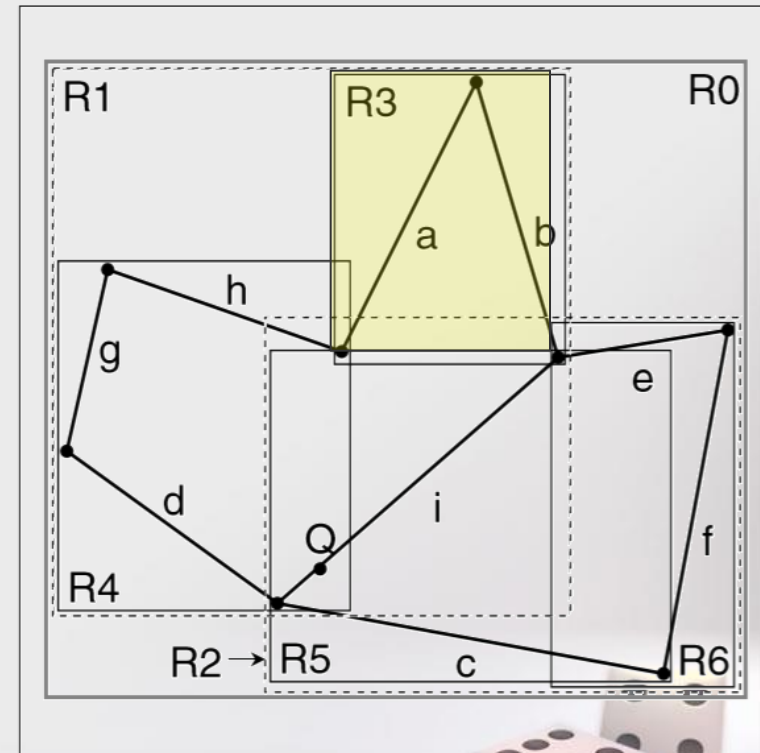
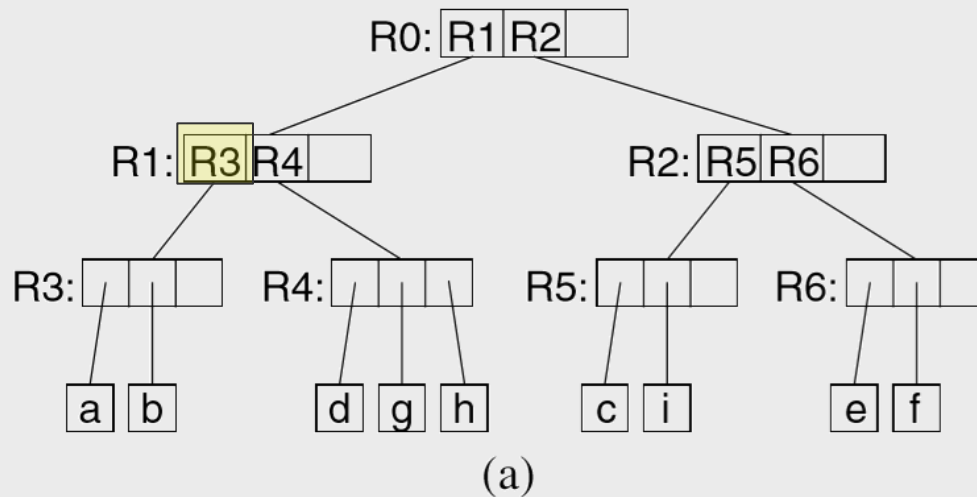
Podem existir **interseções** entre as várias regiões

(b)



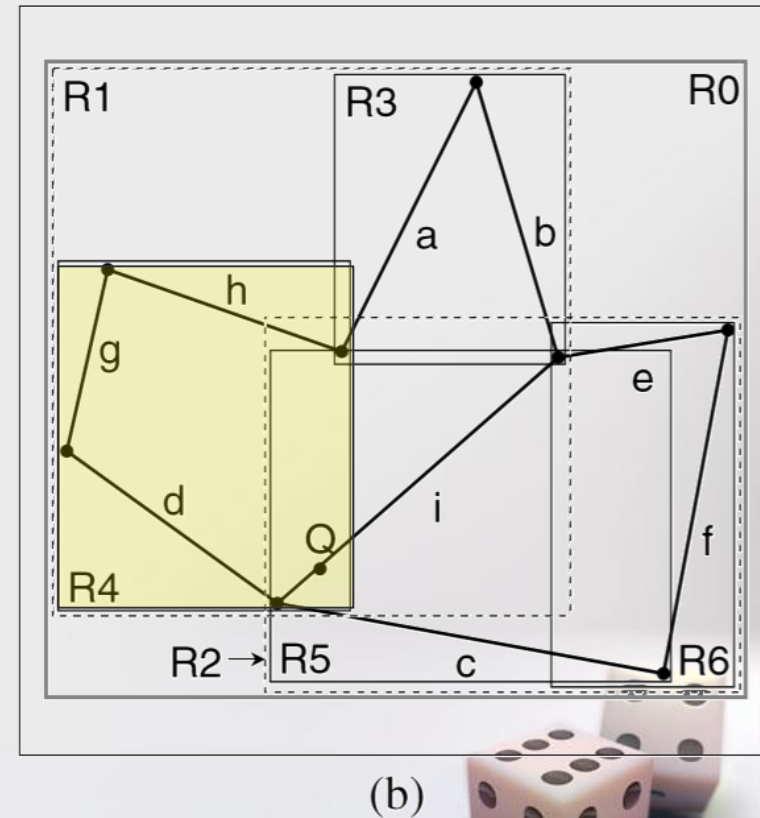
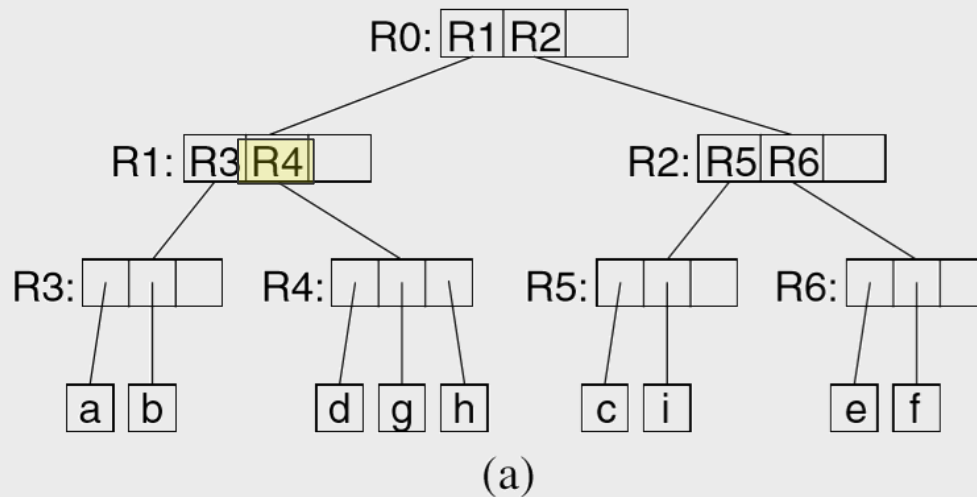
R-Tree

- As R-Tree seguem uma estrutura análoga à das B-Tree



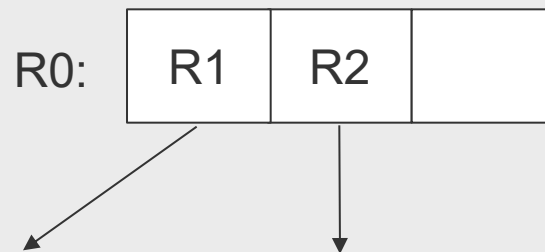
R-Tree

- As R-Tree seguem uma estrutura análoga à das B-Tree



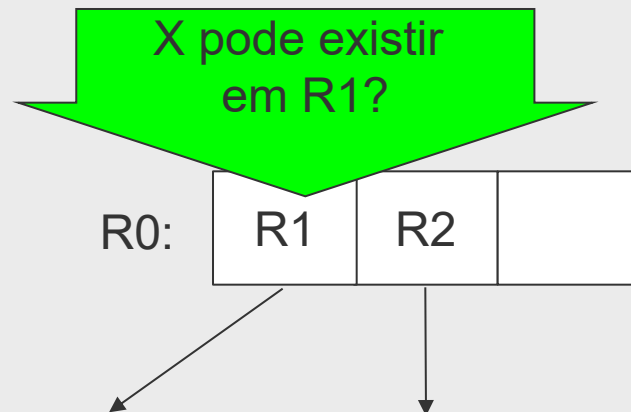
Pesquisa na R-Tree

- A pesquisa de informação num R-Tree é feita verificando se cada uma das regiões pode ou não conter o ponto de interesse:
 - **Exemplo:** *Verifica se existe X em R_0*



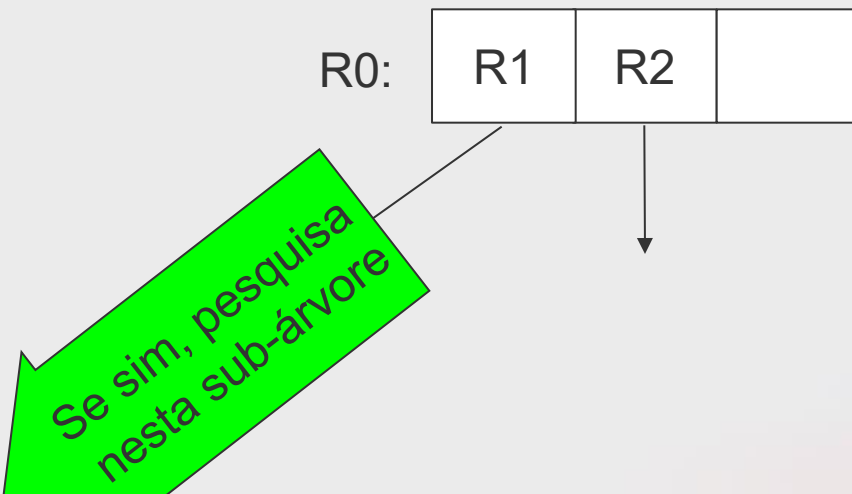
Pesquisa na R-Tree

- A pesquisa de informação num R-Tree é feita verificando se cada uma das regiões pode ou não conter o ponto de interesse:
 - **Exemplo:** *Verifica se existe X em R_0*



Pesquisa na R-Tree

- A pesquisa de informação num R-Tree é feita verificando se cada uma das regiões pode ou não conter o ponto de interesse:
 - **Exemplo:** *Verifica se existe X em R_0*



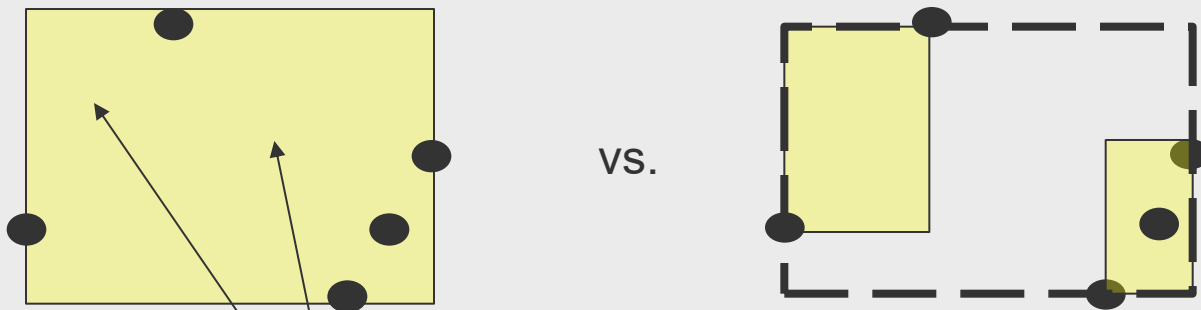
Pesquisa na R-Tree

- A pesquisa de informação num R-Tree é feita verificando se cada uma das regiões pode ou não conter o ponto de interesse:
 - **Exemplo:** *Verifica se existe X em R_0*



R-Tree

- A eficácia da R-Tree depende:
 - *Da sua capacidade de discriminar entre espaço vazio e espaço ocupado*



Pesquisas por pontos nestas localizações obrigam à análise de todos os pontos da região.



R-Tree

- A eficácia da R-Tree depende:
 - *Da sua capacidade de discriminar entre espaço vazio e espaço ocupado*

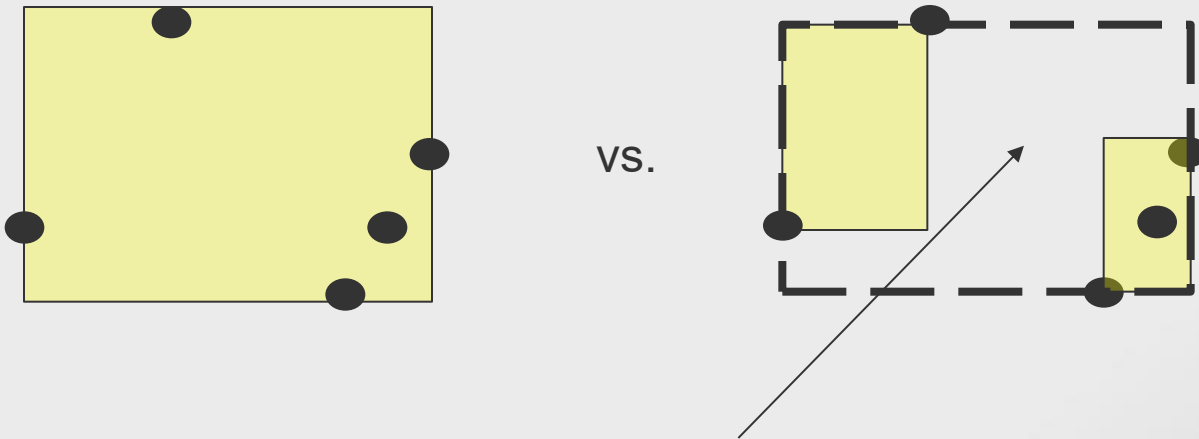


Uma pesquisa nesta localização só vai percorrer 2 pontos.



R-Tree

- A eficácia da R-Tree depende:
 - *Da sua capacidade de discriminar entre espaço vazio e espaço ocupado*

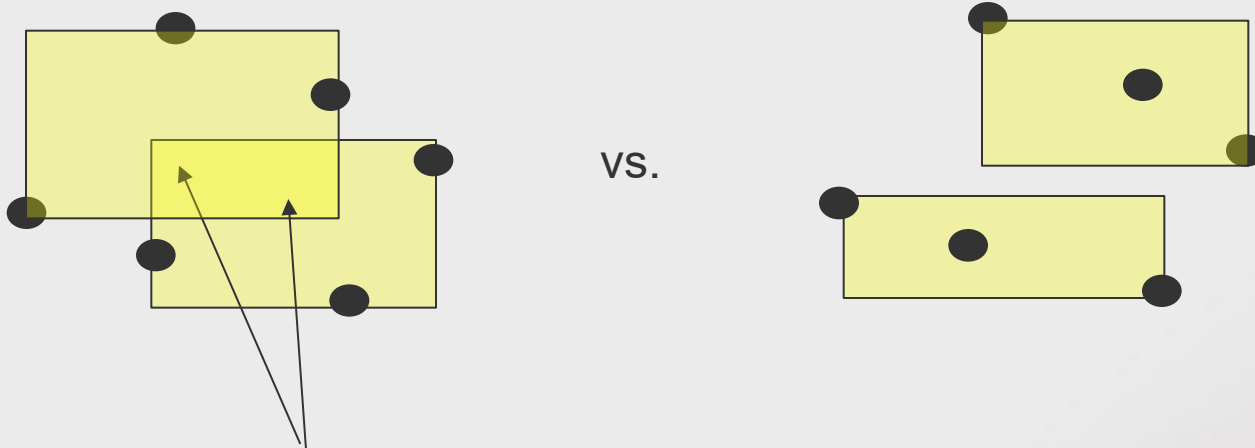


Uma pesquisa nesta localização falha sem analisar pontos.



R-Tree

- A eficácia da R-Tree depende:
 - *Da minimização das sobreposições*



Uma pesquisa nestas localizações abrange ambas as regiões e por isso implica analisar todos os 6 pontos



R-Tree

- A eficácia da R-Tree depende:
 - *Da minimização das sobreposições*

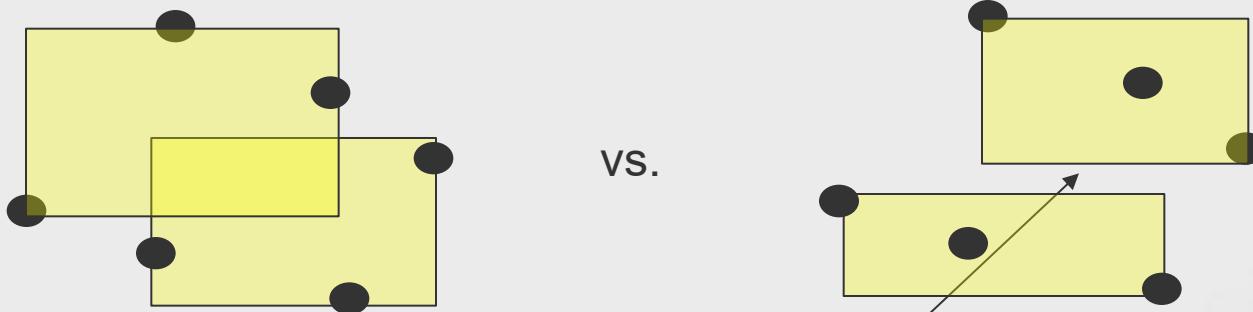


Uma pesquisa nesta localização irá necessitar de percorrer apenas uma região (3 pontos)



R-Tree

- A eficácia da R-Tree depende:
 - *Da minimização das sobreposições*



Uma pesquisa nesta localização irá falhar sem aceder a pontos.



Eficiência da R-Tree

- Sintetizando, o objetivo nas R-Trees é:
 - *Minimizar a cobertura e as sobreposições das regiões.*
 - *Estes objetivos são tidos em conta nos processos de criação e atualização da árvore (que não vamos analisar aqui em detalhe)*
 - *Há outras estruturas que tentam resolver estas questões.*

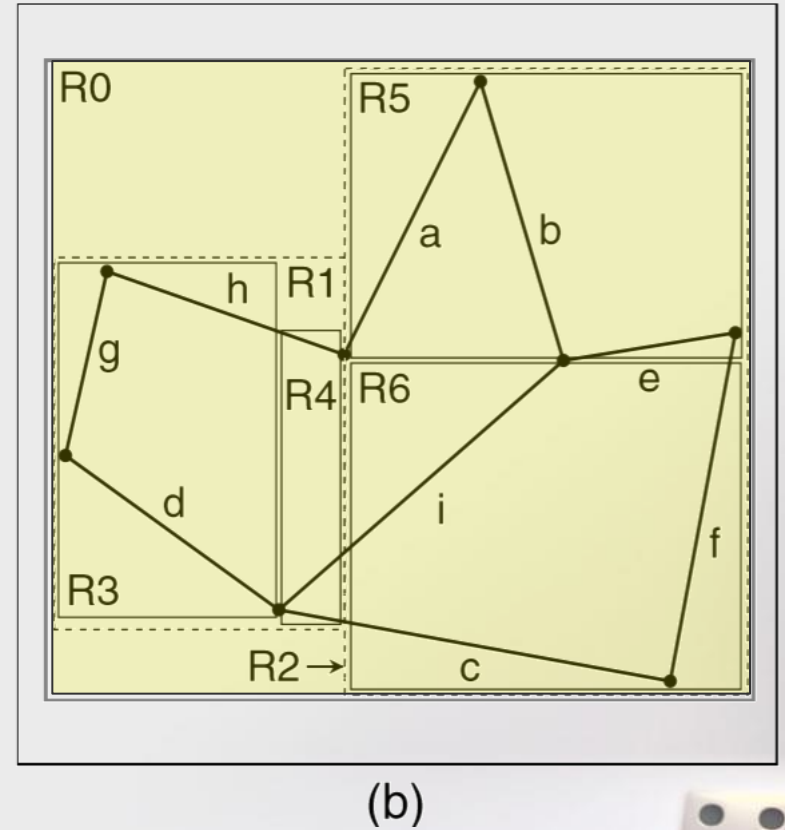
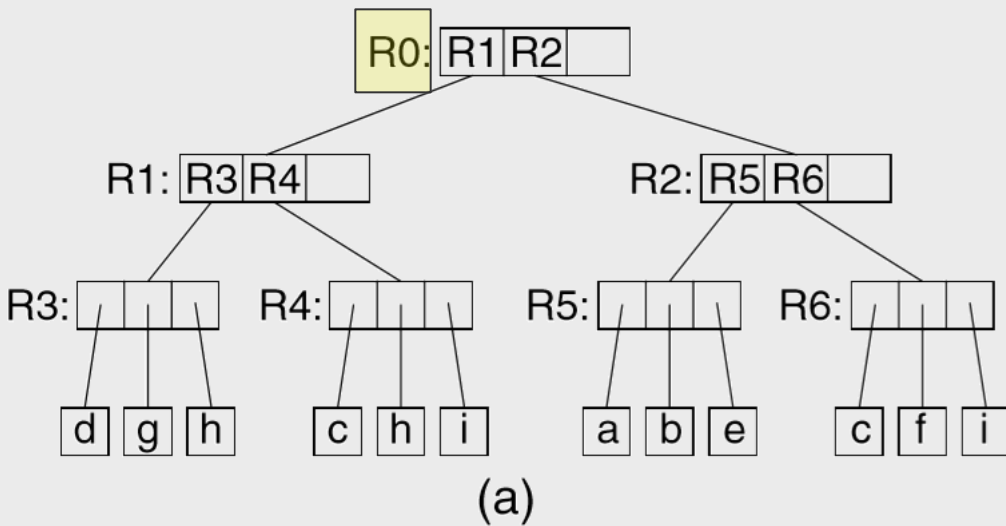


R⁺-Tree

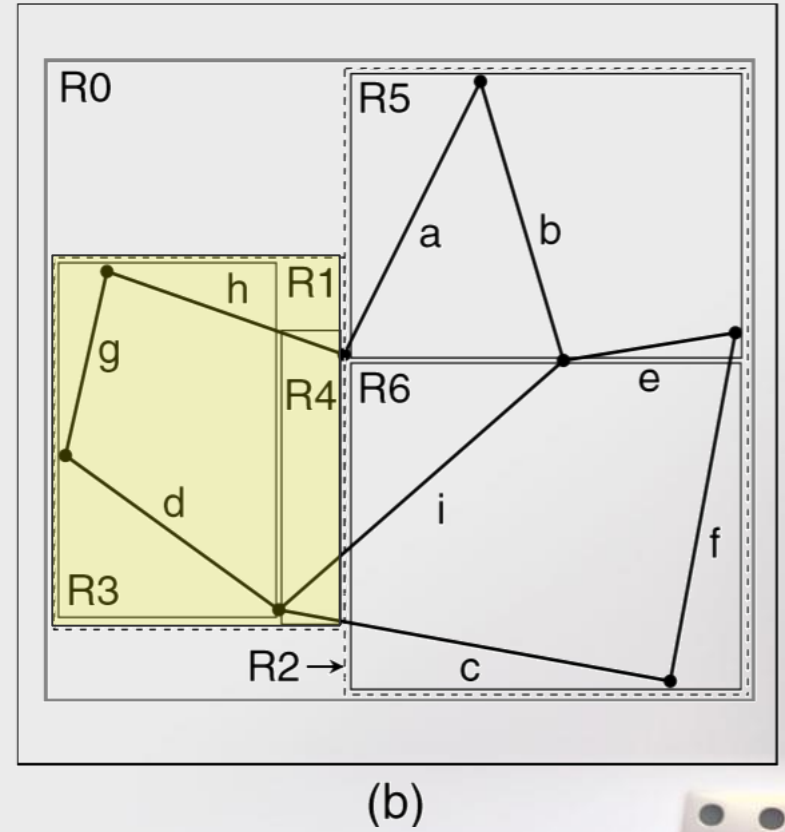
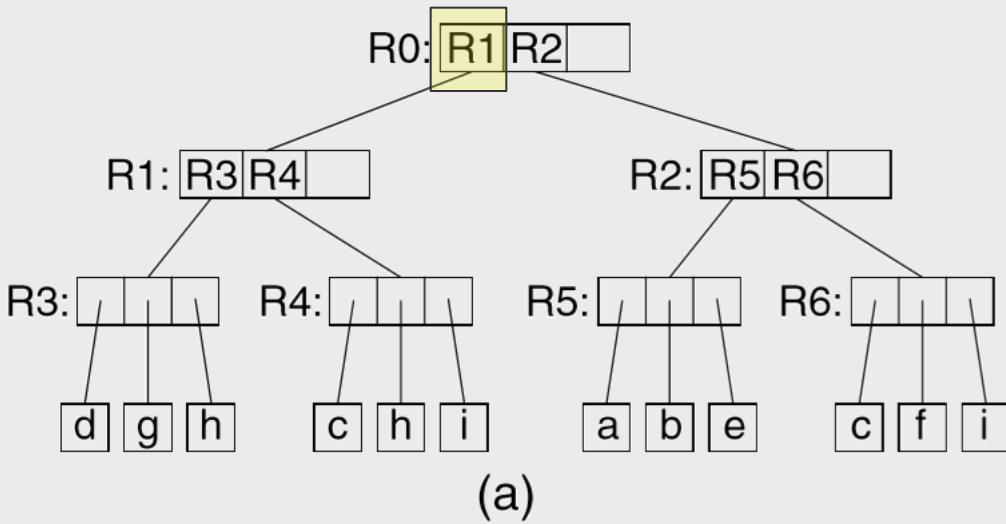
- Na *R⁺-Tree*, a decomposição de uma célula resulta em células **disjuntas**, evitando assim o problema de ineficiência que pode afetar as *R-Trees* normais.
 - *Esta propriedade pode ser violada nas folhas em casos degenerados (p.ex. linhas)*
- Como consequência desta restrição, a mesma informação pode aparecer em mais do que uma folha e a árvore torna-se mais profunda.
 - *As pesquisas são, no entanto mais rápidas.*



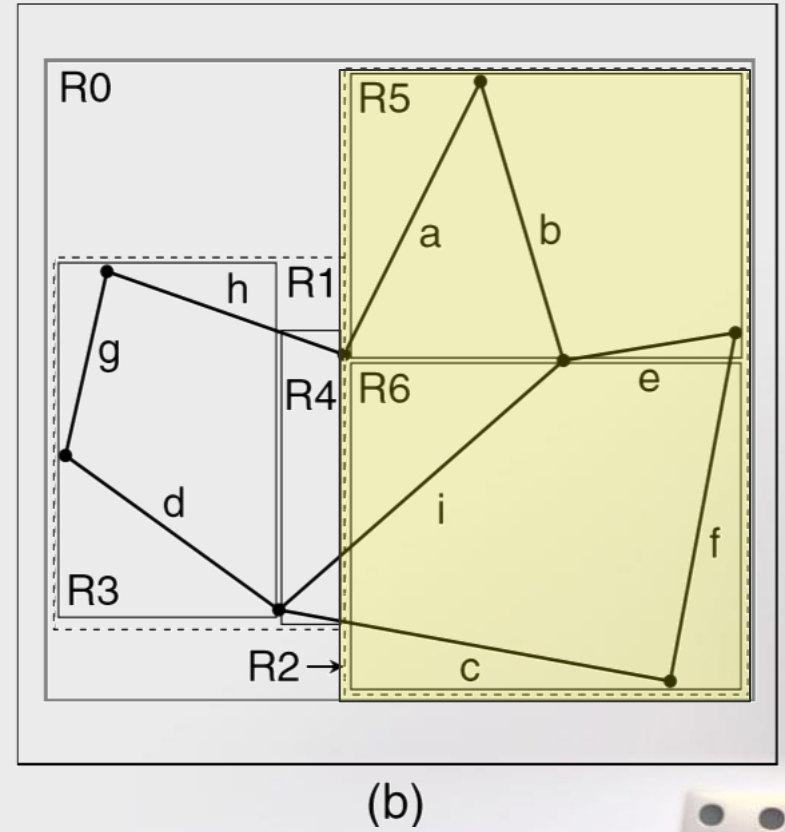
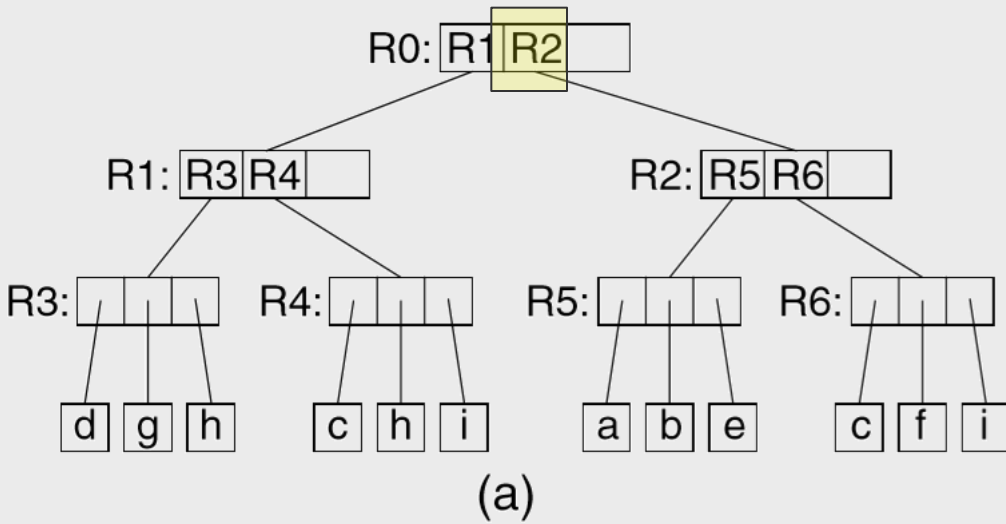
R⁺-Tree



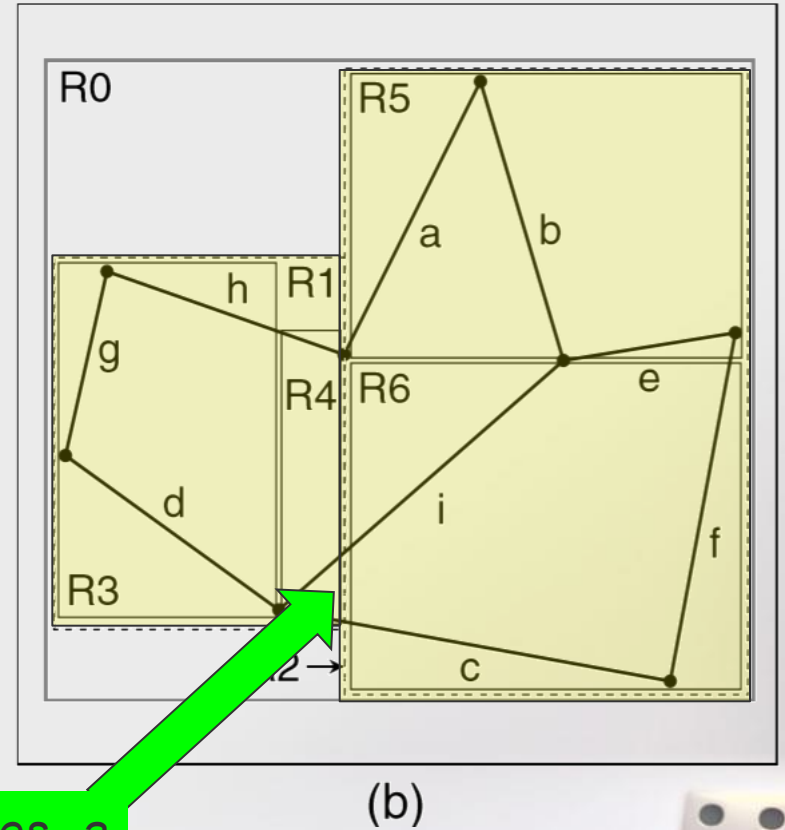
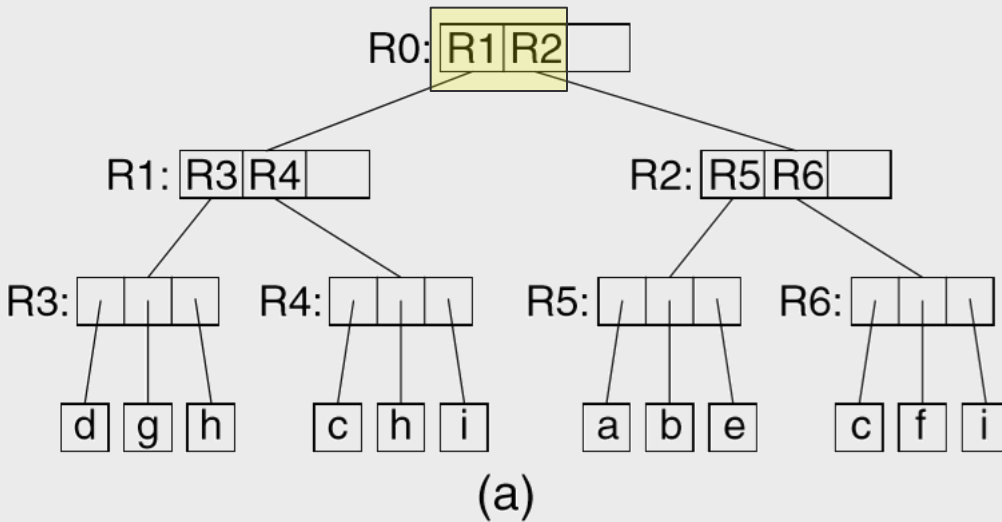
R⁺-Tree



R⁺-Tree



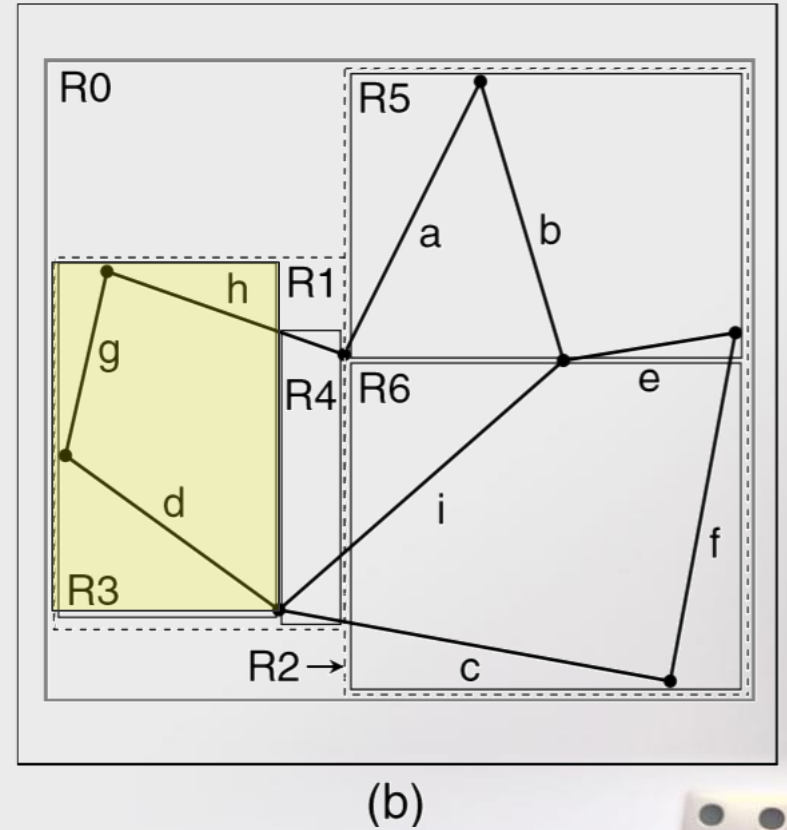
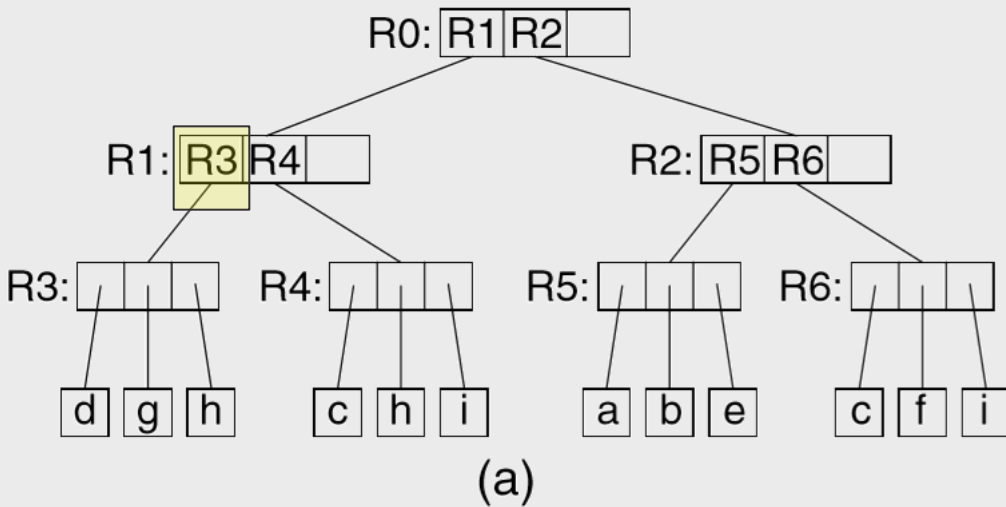
R⁺-Tree



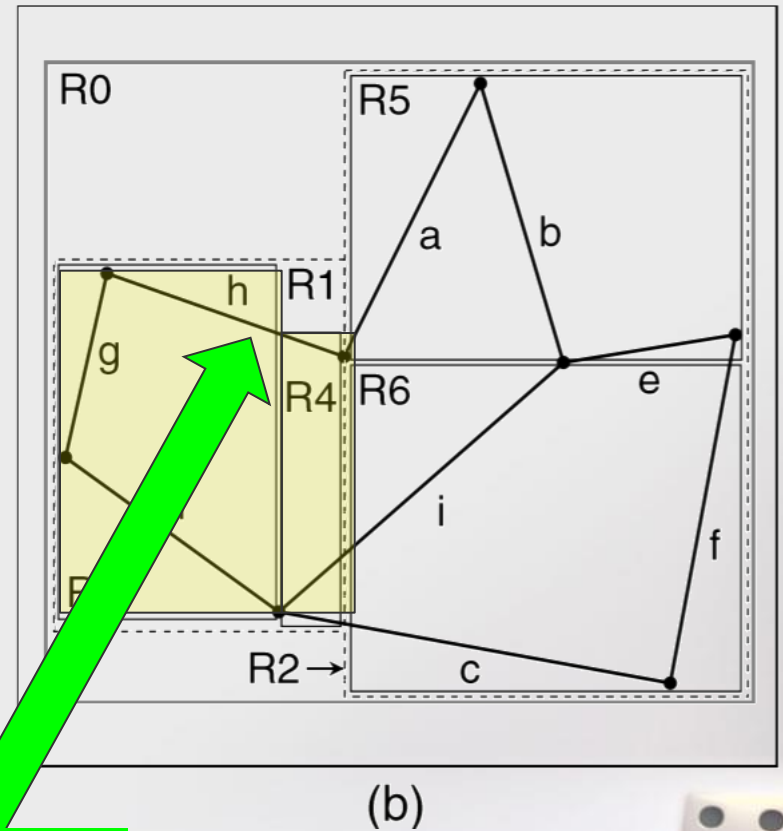
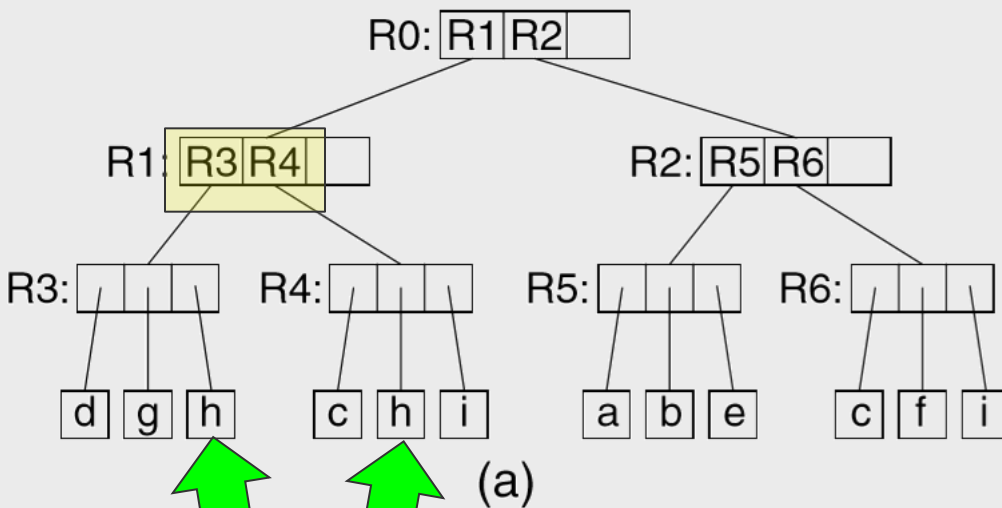
Não há sobreposições, a decomposição é disjunta.



R⁺-Tree



R⁺-Tree



A reta *h* pertence **tanto** a *R3* como a *R4*.
Ir  por isso aparecer em multiplas folhas.



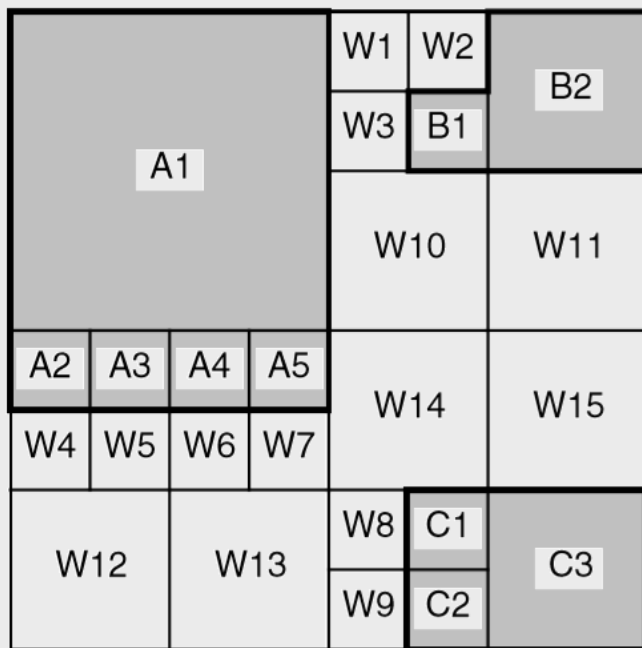
Representação de Regiões

- Há diferentes formas de representar regiões:
 - *Através da sua fronteira*
 - *Através do seu interior*
- Vamos ver muito brevemente algumas opções para a segunda abordagem:
 - *Os objetos são construídos através de coleções de objetos primitivos*
 - *Vamos considerar células homogêneas, disjuntas de área/volume unitário e ortogonais.*

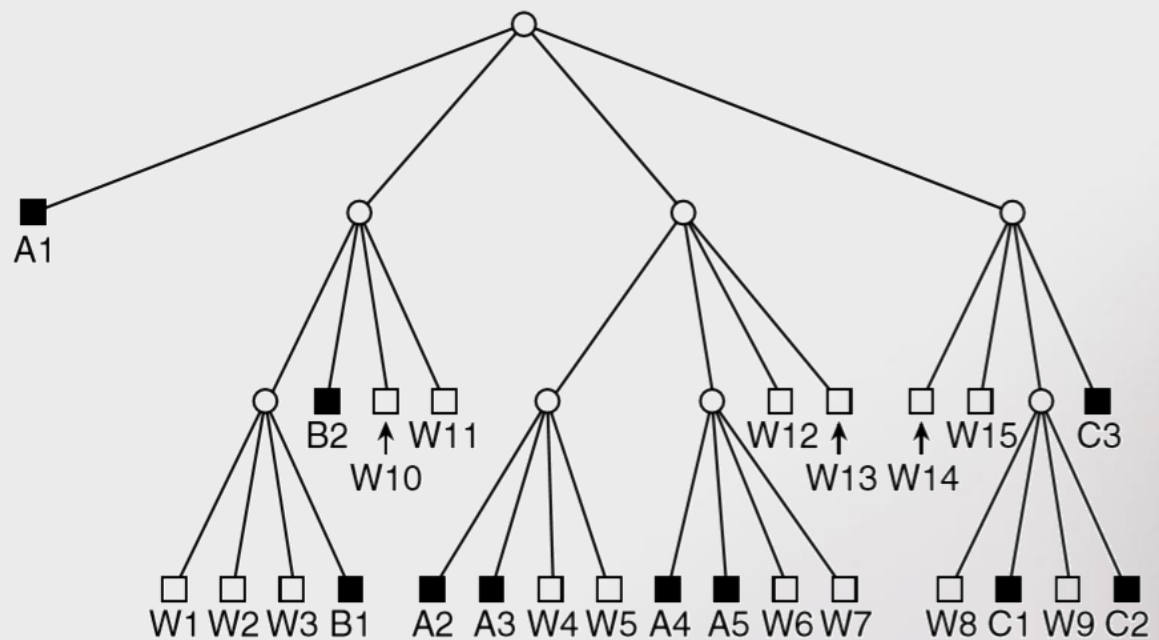


Quadtrees de Regiões

- Perfeitamente análoga a Quadtrees de Regiões, mas cada nodo representa toda uma região e não apenas um ponto



(a)

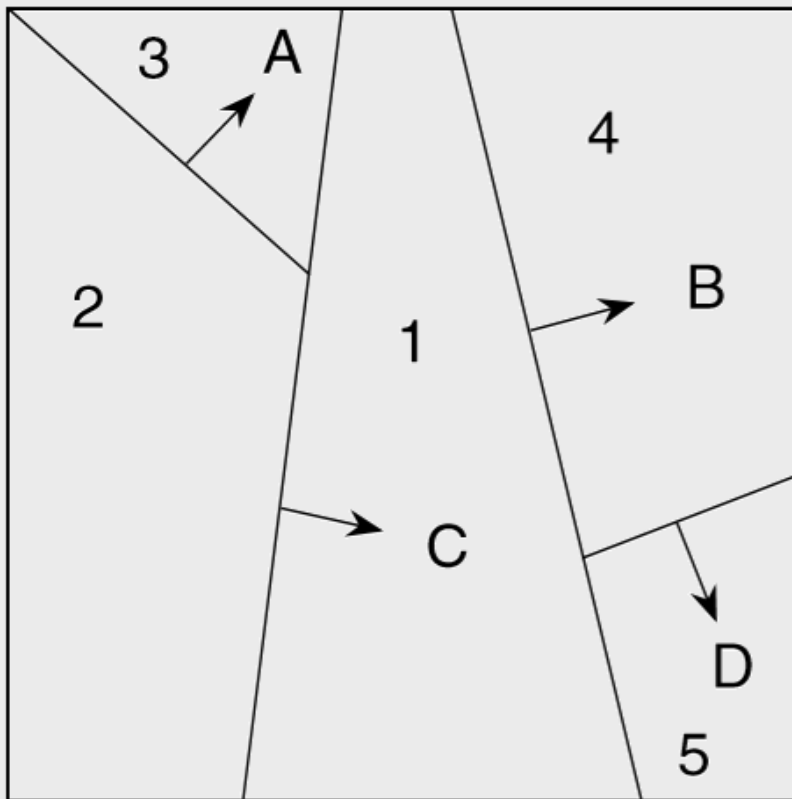


(b)

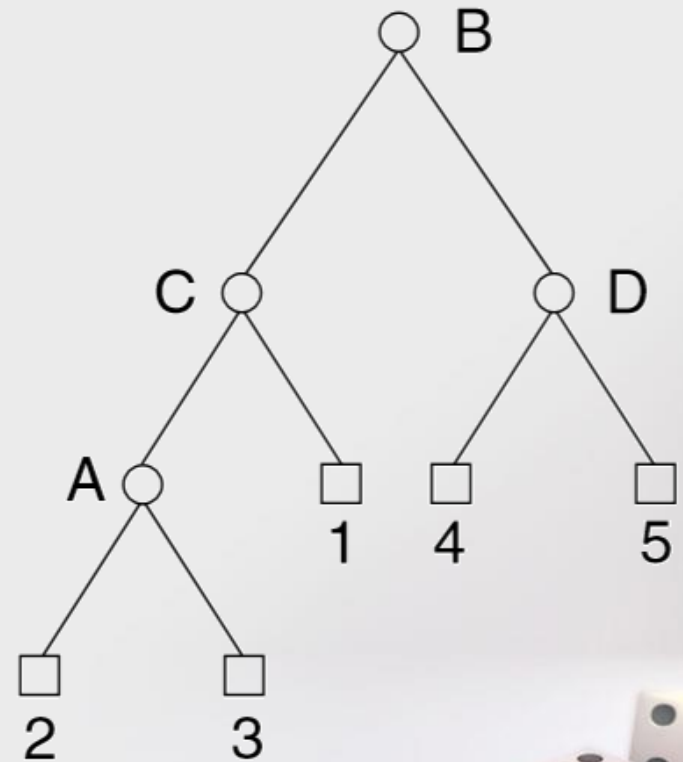


BSP-Trees

- Binary Space Partitioning Tree:



(a)

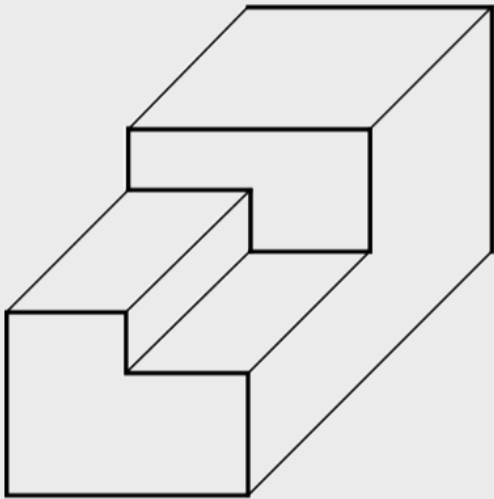


(b)

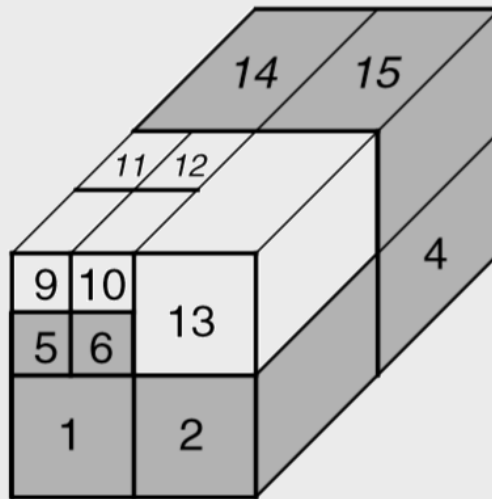


Regiões Tridimensionais

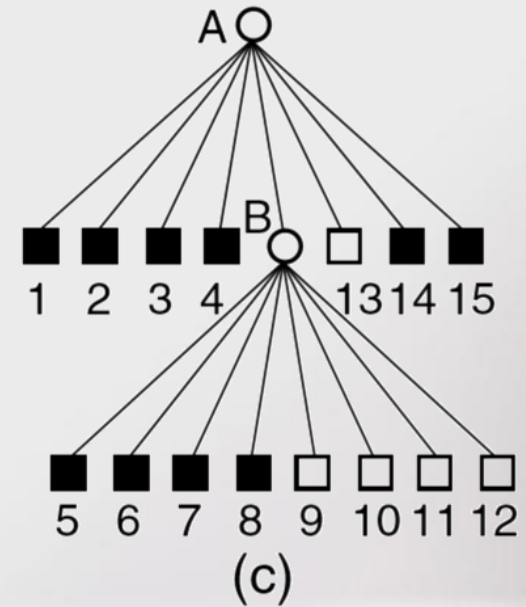
- Region Octree



(a)



(b)



(c)



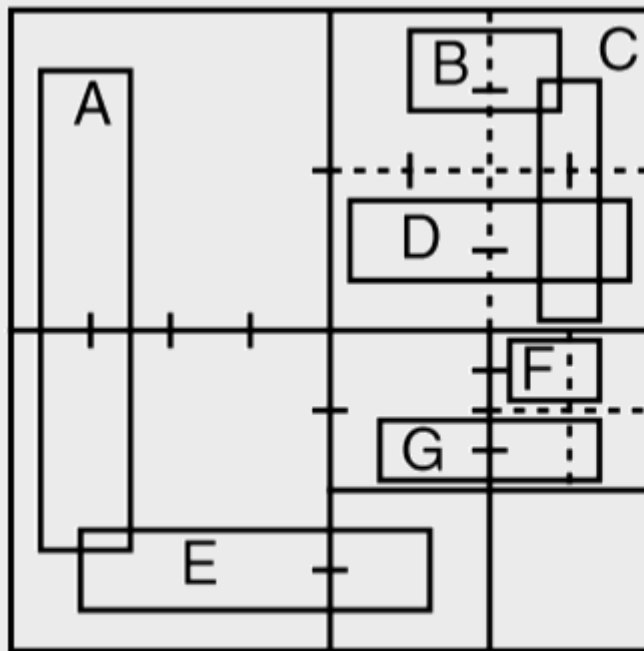
Armazenamento de Retangulos

- Problema “híbrido” entre armazenamento de pontos e regiões. Algumas abordagens são:
 - *Mapear o retângulo como um único ponto de maior dimensão.*
 - *Usar uma variação de Quad-trees conhecida como MX-CIF.*
 - *As regiões das R-Trees ou R+-Trees são retângulos, pelo que essas estruturas são também naturalmente adequadas para armazenar retângulos.*

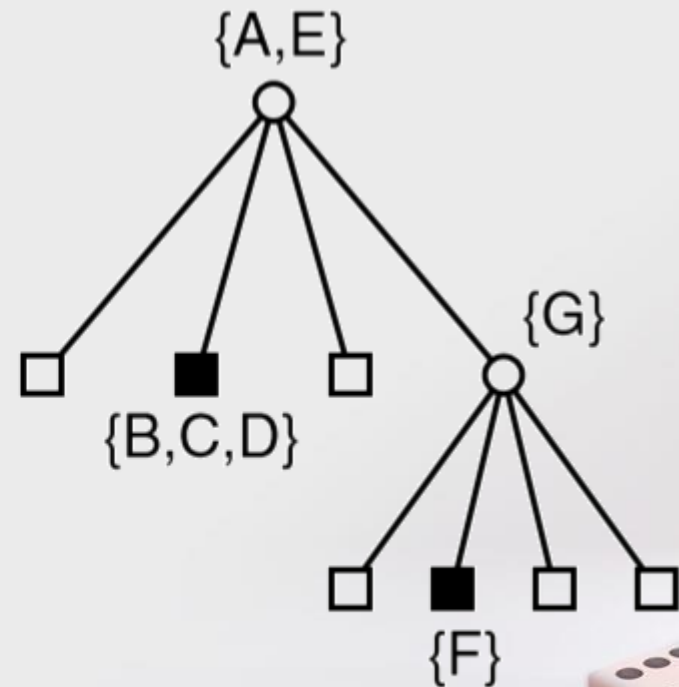


MX-CIF

- Cada nodo da Quad-tree armazena informação sobre os retângulos que estão ***completamente*** contidos na área resultante.



(a)

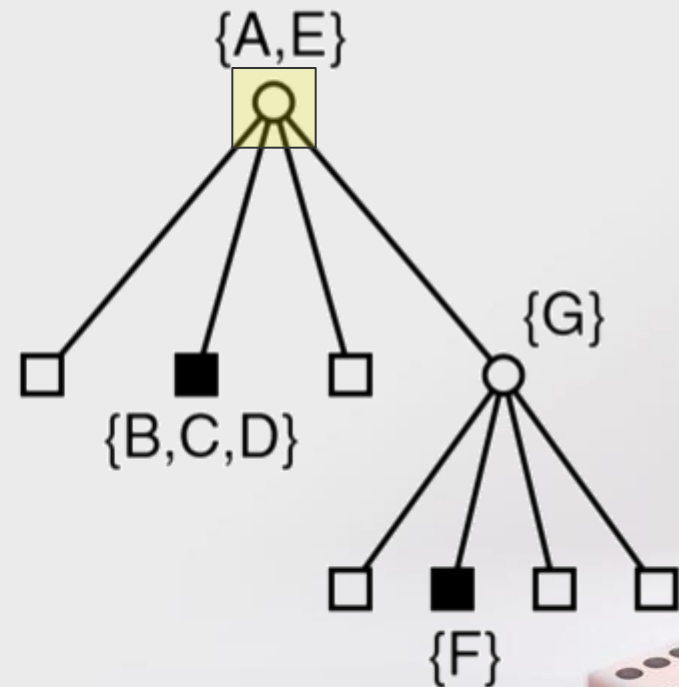
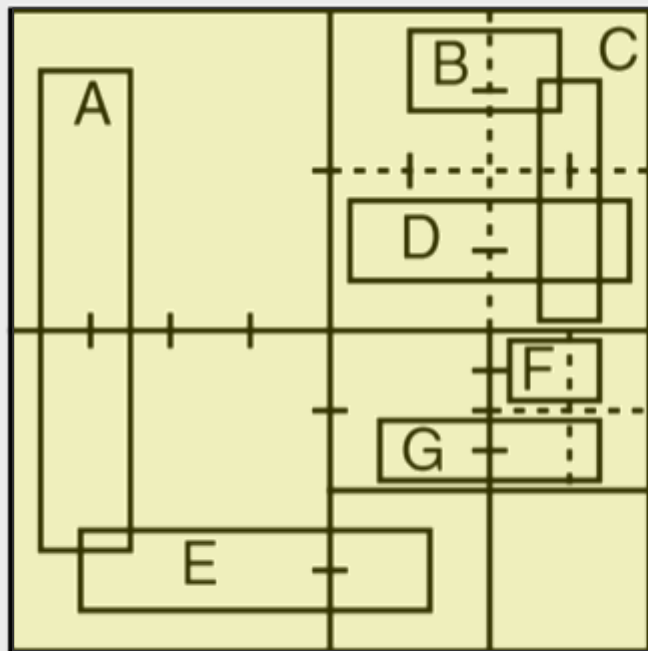


(b)



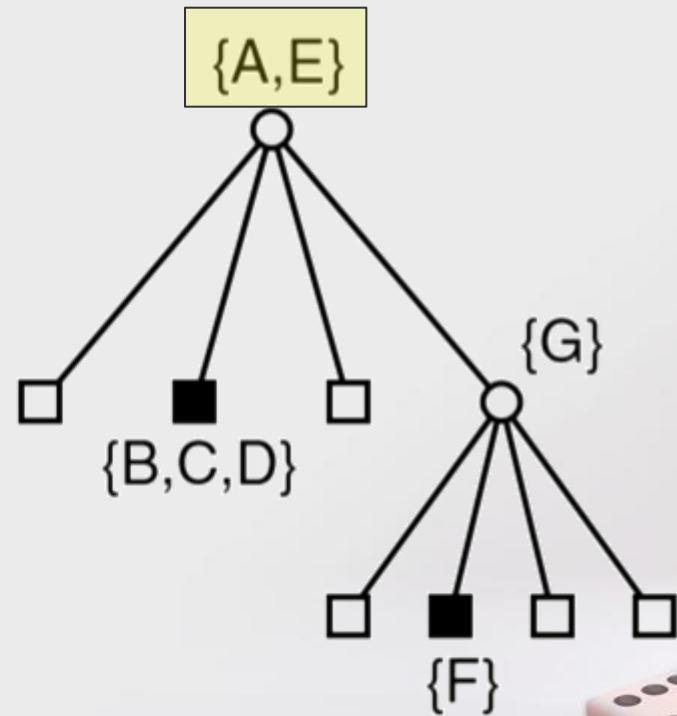
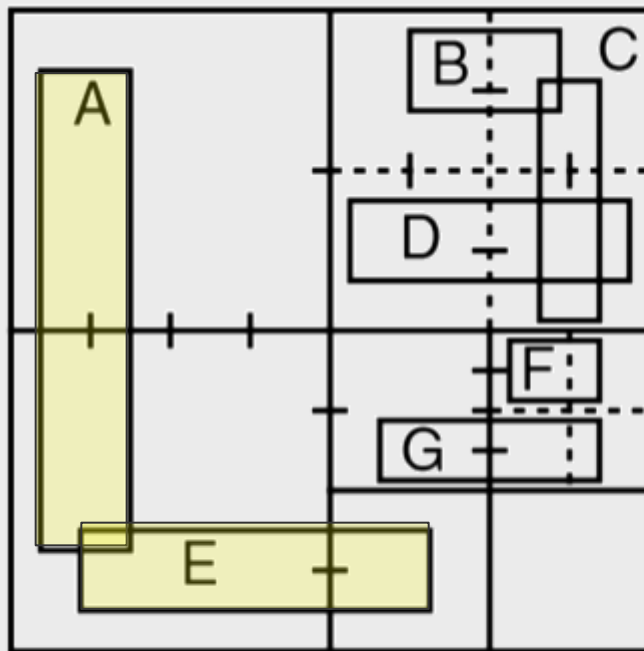
MX-CIF

- Cada nodo da Quad-tree armazena informação sobre os retângulos que estão **completamente** contidos na área resultante (*e não apenas num dos nodos descendentes*)



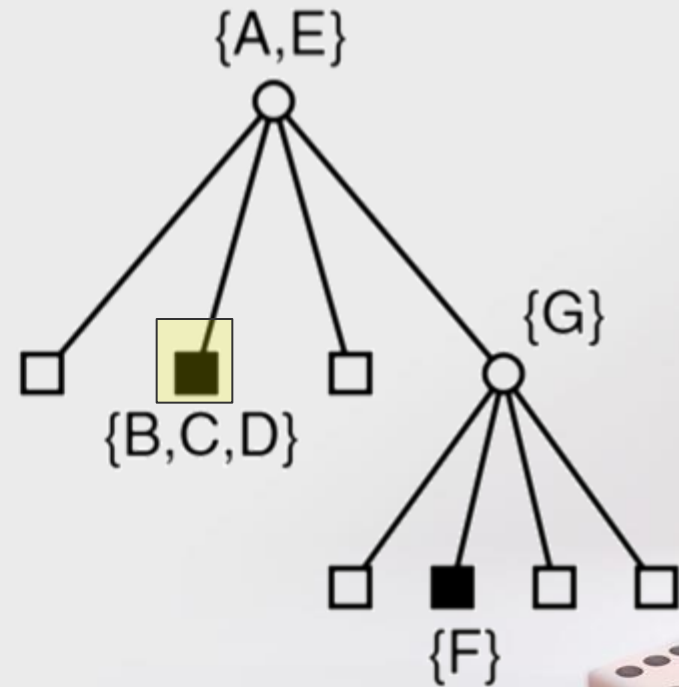
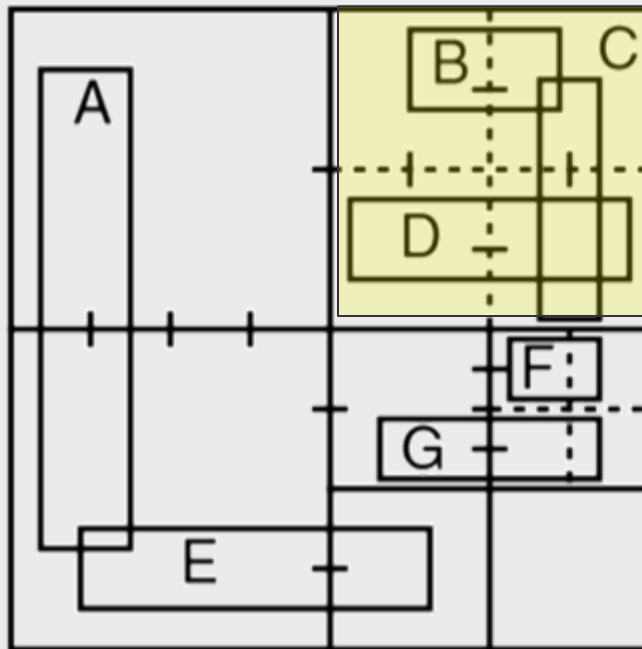
MX-CIF

- Cada nodo da Quad-tree armazena informação sobre os retângulos que estão **completamente** contidos na área resultante (*e não apenas num dos nodos descendentes*)



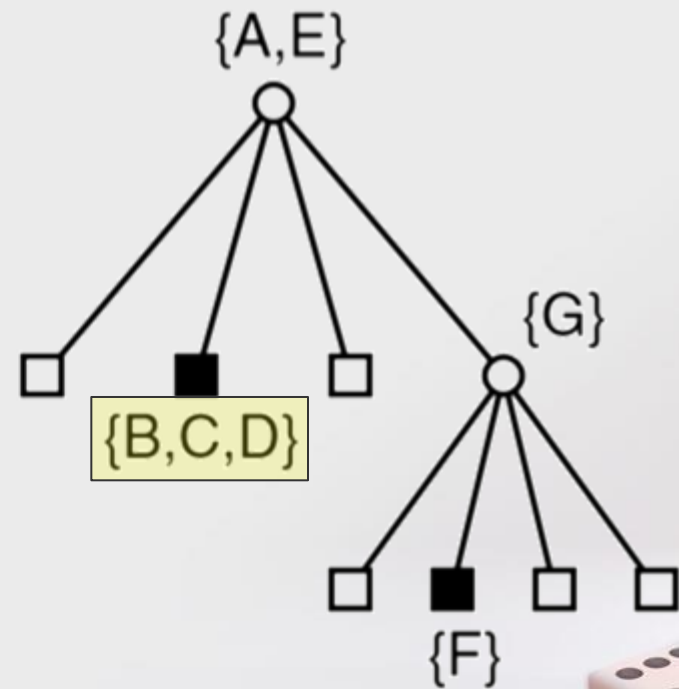
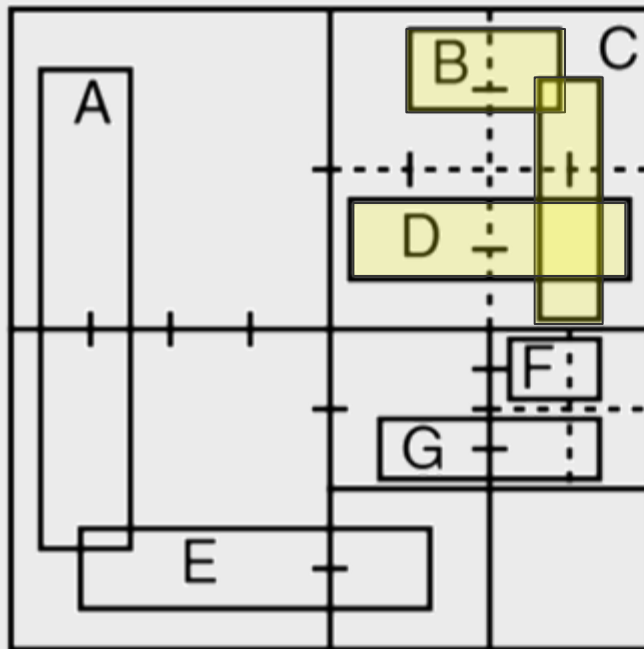
MX-CIF

- Cada nodo da Quad-tree armazena informação sobre os retângulos que estão **completamente** contidos na área resultante (*e não apenas num dos nodos descendentes*)



MX-CIF

- Cada nodo da Quad-tree armazena informação sobre os retângulos que estão **completamente** contidos na área resultante (*e não apenas num dos nodos descendentes*)



MX-CIF

- Dado que cada nodo da Quad-tree pode armazenar vários retângulos, são também frequentemente utilizadas árvores auxiliares em cada nodo para representar numa escala linear a intersecção desses retângulos com os eixos:

